

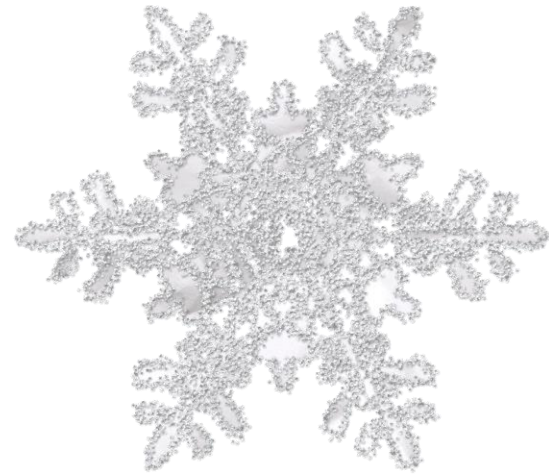
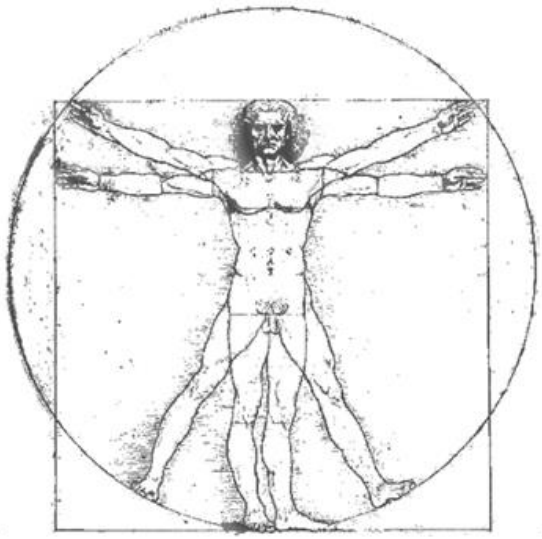
Geometric Deep Learning

CPSC483: Deep Learning on Graph-Structured Data

Rex Ying

Symmetry in Nature

Nature is *highly symmetric* because it is easier to encode / store.

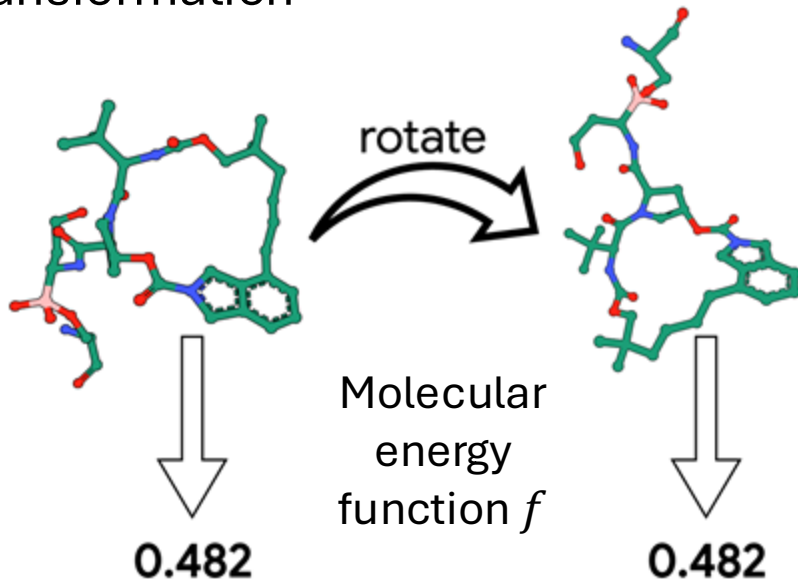


Defn: *Symmetries* are transformations of objects by which certain properties/aspects of the object **stay the same**.

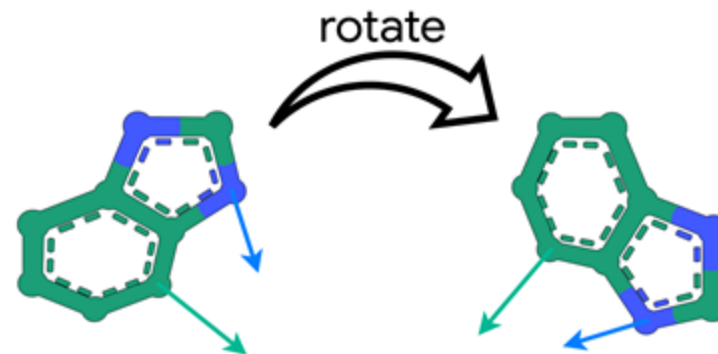
Physical Symmetries

If we transform an object such that its property ...

... stays the same, we say that **scalar** quantity is **invariant** to the transformation



... changes by an equivalent amount, we say that **vector** quantity is **equivariant** to the transformation



Why Modeling Symmetry?

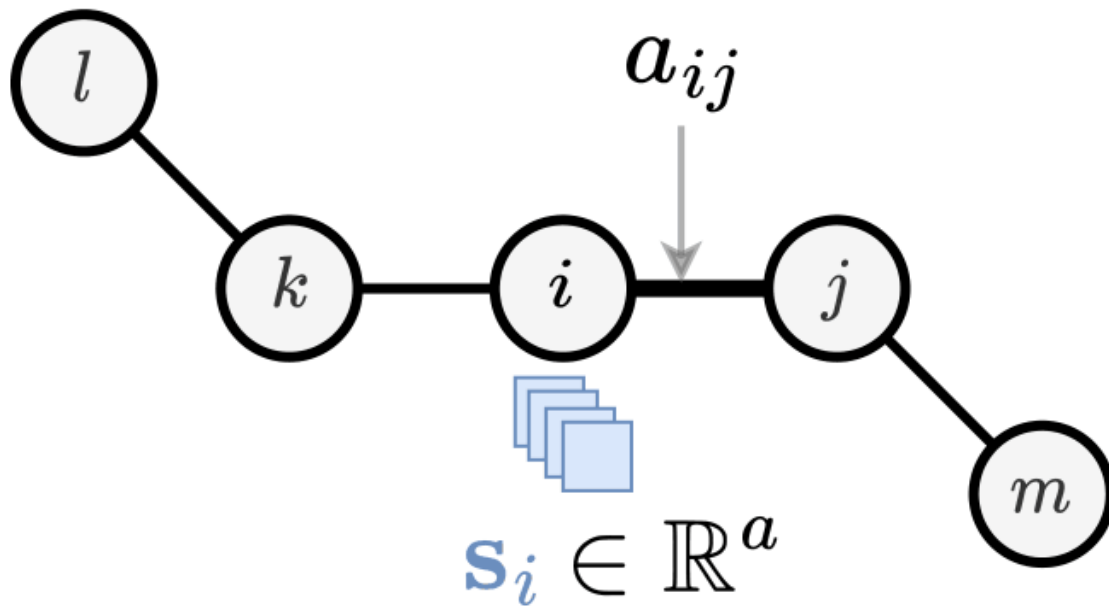
- Symmetries are present across scientific domains (physics, molecular chemistry, fluid dynamics)
- Certain laws of nature must be obeyed (conservation of energy)
- Certain quantities in nature are invariant or equivariant (velocity, solubility, energy, ...)

Equivariance to rigid-body motions is a key symmetry

(3D rotations, reflections, translations)

Recap: Graphs

Graph generally contains a set of nodes and edges. The **geometric** shape or structure of the graph has no meaning.



$$G = (\mathbf{A}, \mathbf{S})$$

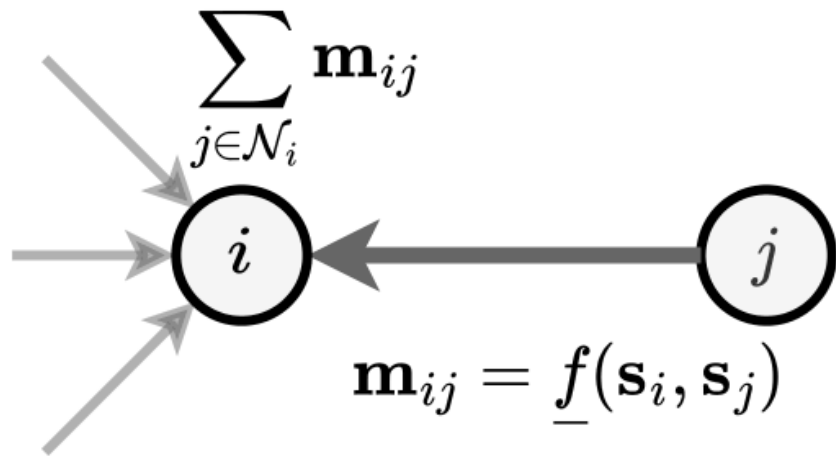
2D adjacency
 $\in [0,1]^{n \times n}$

Node features
 $\in \mathbb{R}^{n \times a}$

a is the embedding dimension for scalar node features, making $\mathbf{s}$ a list.

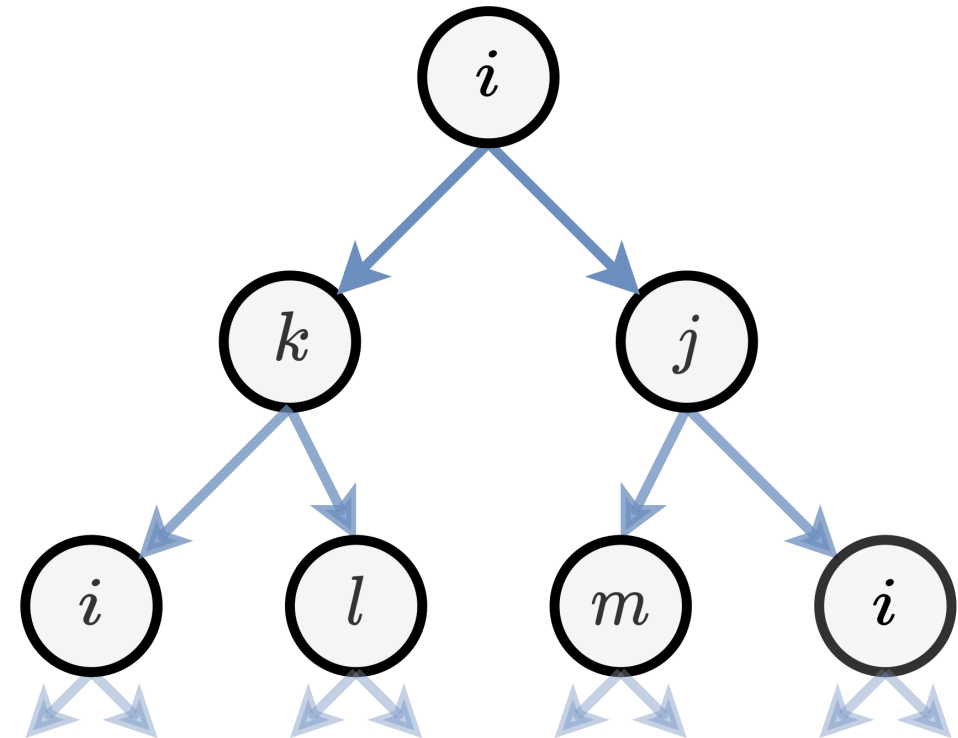
Recap: Message Passing on Graphs

Messages are aggregated across local neighborhoods. This creates an expanding **computation graph** of nodes and their (distant) neighbors.



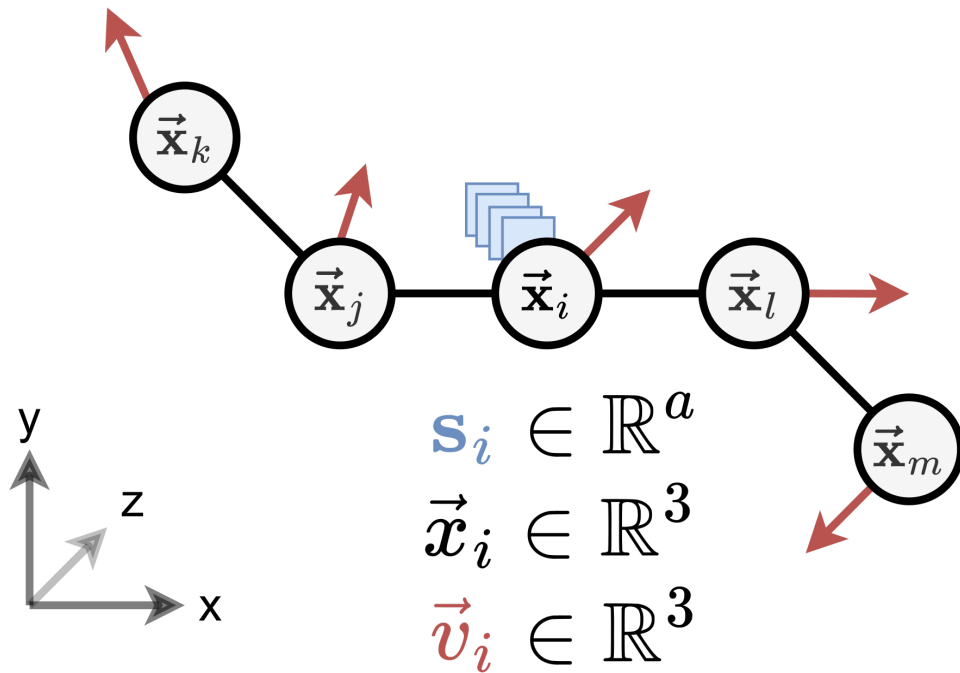
$$\mathbf{m}_i^{(t)} = \text{Agg} \left(\left\{ f_{\theta} \left(\mathbf{s}_i^{(t)}, \mathbf{s}_j^{(t)} \right) \mid j \in \mathcal{N}_i \right\} \right)$$

$$\mathbf{s}_i^{(t+1)} = \text{Upd} \left(\mathbf{s}_i^{(t)}, \mathbf{m}_i^{(t)} \right)$$



Geometric Graphs

Nodes are embedded in Euclidean space with geometric attributes.



Regular MPNNs cannot handle such graphs in a symmetry-aware fashion.

$$G = (\mathbf{A}, \mathbf{S}, \vec{\mathbf{X}}, \vec{\mathbf{V}})$$

Adjacency
 $\in [0,1]^{n \times n}$

Scalar features
 $\in \mathbb{R}^{n \times a}$

Coordinates
 $\in \mathbb{R}^{n \times 3}$

Vector features
 $\in \mathbb{R}^{n \times c \times d}$

Adjacency
usually
omitted.

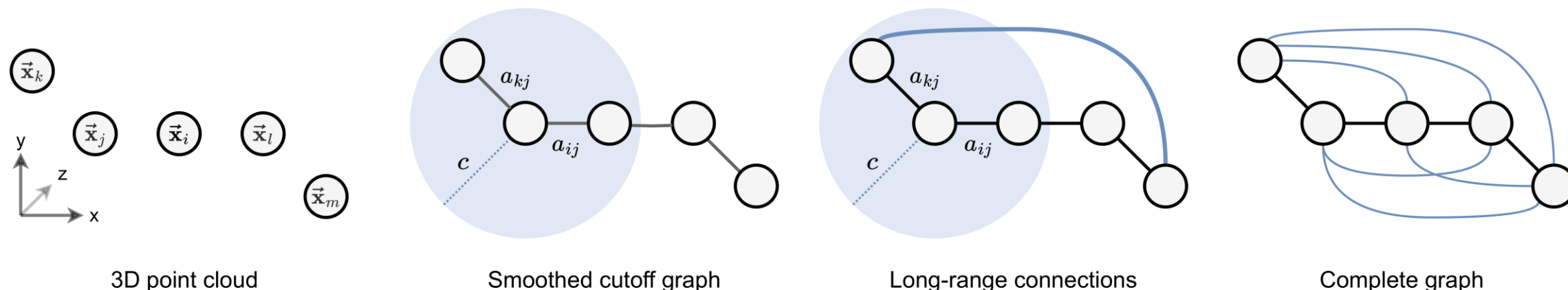
Drawing “Edges”

In domains like chemistry, the concept of a bond is a simplification.

→ All atoms exert forces on each other which are either **strong** or **weak** based on *proximity*.

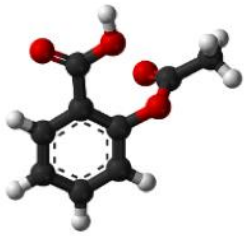
→ Regular graphs have a binary notion of edges (0/1 entry in adjacency matrix).

To better capture proximity-based edges, **how we draw edges is a design choice.**

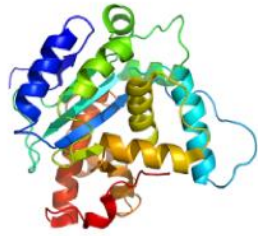


For most applications, **sparse radial cutoff graphs** are good enough.

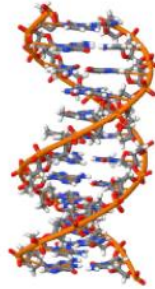
Systems with symmetry and relational structure



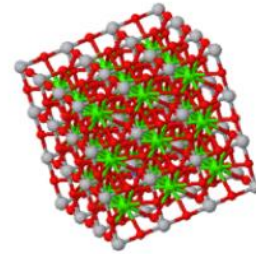
Small
Molecules



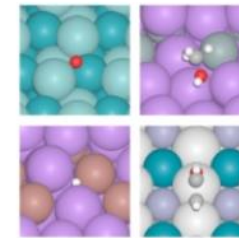
Proteins



DNA/RNA



Inorganic
Crystals



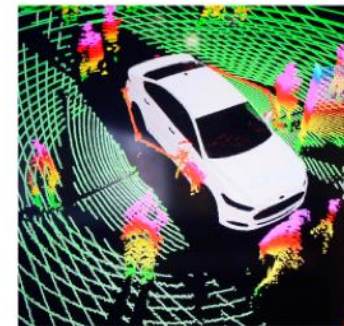
Catalysis
Systems



Transportation &
Logistics



Robotic
Navigation

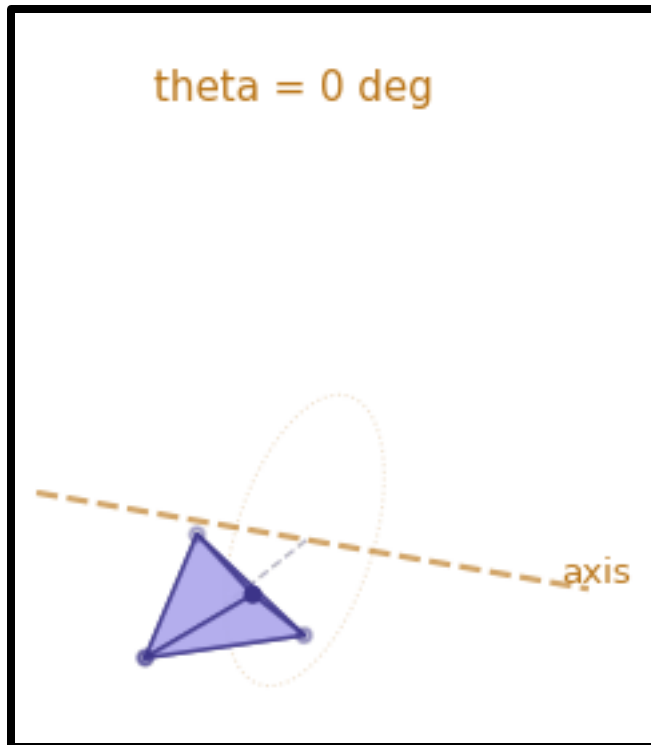


3D Computer
Vision

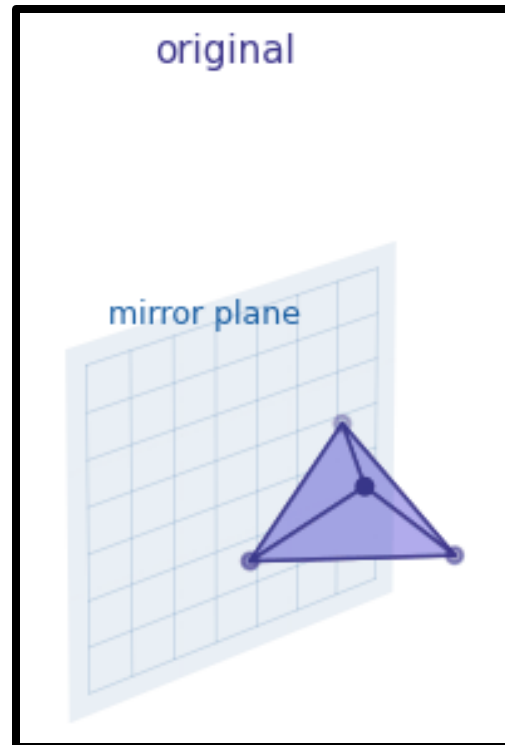
Key Rigid-body Motions

3D rigid-body motions include,

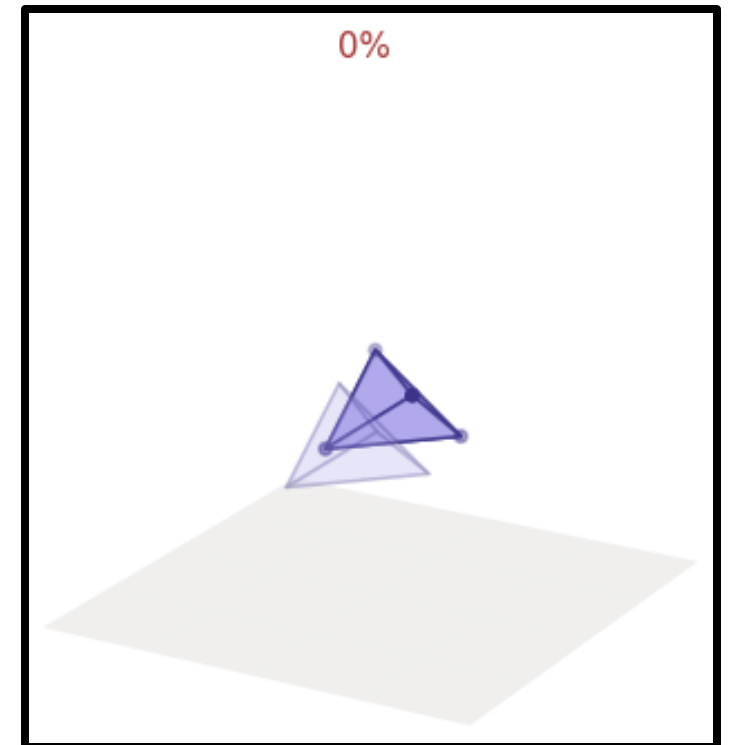
Rotations



Reflections



Translations



Group Theory

A *Group* $\mathcal{G}$ is a set with a binary operation, $\bullet$.

Its elements follow a few axioms (rules).

1. For $g, h \in \mathcal{G}$, $g \bullet h$ is also in $\mathcal{G}$
2. Associativity: $(g \bullet h) \bullet t = g \bullet (h \bullet t)$
3. There must be a identity element, e , that does nothing, $g \bullet e = g$
4. For every element g , there must be an inverse h such that $g \bullet h = e$

Anything that fits this description can be called a group.

Example: Integers form a group $(\mathbb{Z}, +)$

Rigid-body Motions and Group Theory

In 3D, different combinations of rigid-body motions **form different groups** with special names.

- Special Orthogonal Group, $SO(3)$: **rotations** only
- Orthogonal Group, $O(3)$: **rotations** and **reflections**
- $T(3)$: **translations** only
- Special Euclidean Group, $SE(3)$: **rotations** and **translations**
- Euclidean Group, $E(3)$: **rotations**, **reflections**, **translations**



Group Representations

“150° about the x-axis”, “34° about the y-axis” $\in \mathbf{SO}(3)$.

These describe motions in a vague abstract way.

What do they look like on a computer?

→ We use *group representations*

→ These are just 3x3 real-valued matrices

→ We can use these matrices in computations

$$R_x(\theta) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta \\ 0 & \sin \theta & \cos \theta \end{pmatrix}$$

$$R_y(\theta) = \begin{pmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{pmatrix}$$

$$R_z(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Group Actions in $\mathbb{R}^3$

- A group $\mathcal{G}$ can **act** on another set X
- A 3D rotation acts on a 3D point via **matrix-vector multiplication**

To denote rotating the point p by 130° about the x-axis,

$$\underbrace{\begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos 130^\circ & -\sin 130^\circ \\ 0 & \sin 130^\circ & \cos 130^\circ \end{pmatrix}}_{R_x(130^\circ)} \times \underbrace{\begin{pmatrix} a \\ b \\ c \end{pmatrix}}_p$$

Take Note:

When $\vec{x} \in \mathbb{R}^3$, the rotated **point** is $R_x \vec{x}$. When $\vec{X} \in \mathbb{R}^{n \times 3}$, the rotated **point cloud** is $\vec{X} R_x^\top$.

Composition of Group Actions in $\mathbb{R}^3$

Back-to-back rotations can be composed using matrix-matrix multiplications.

Eg: 1) rotate p by 36° about y -axis, and 2) rotate p by 130° about x -axis:

$$\underbrace{\begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos 130^\circ & -\sin 130^\circ \\ 0 & \sin 130^\circ & \cos 130^\circ \end{pmatrix}}_{R_x(130^\circ)} \times \underbrace{\begin{pmatrix} \cos 36^\circ & 0 & \sin 36^\circ \\ 0 & 1 & 0 \\ -\sin 36^\circ & 0 & \cos 36^\circ \end{pmatrix}}_{R_y(36^\circ)} \times \underbrace{\begin{pmatrix} a \\ b \\ c \end{pmatrix}}_p$$

When $\vec{x} \in \mathbb{R}^3$, the rotated **point** is $R_x R_y \vec{x}$.

When $\vec{X} \in \mathbb{R}^{n \times 3}$, the rotated **point cloud** is $(\vec{X} R_y^\top) R_x^\top$.

Subgroups

A *Subgroup* is a smaller subset of items from a group that also obeys the group axioms. **Rigid-body motions all form subgroups of one another.**

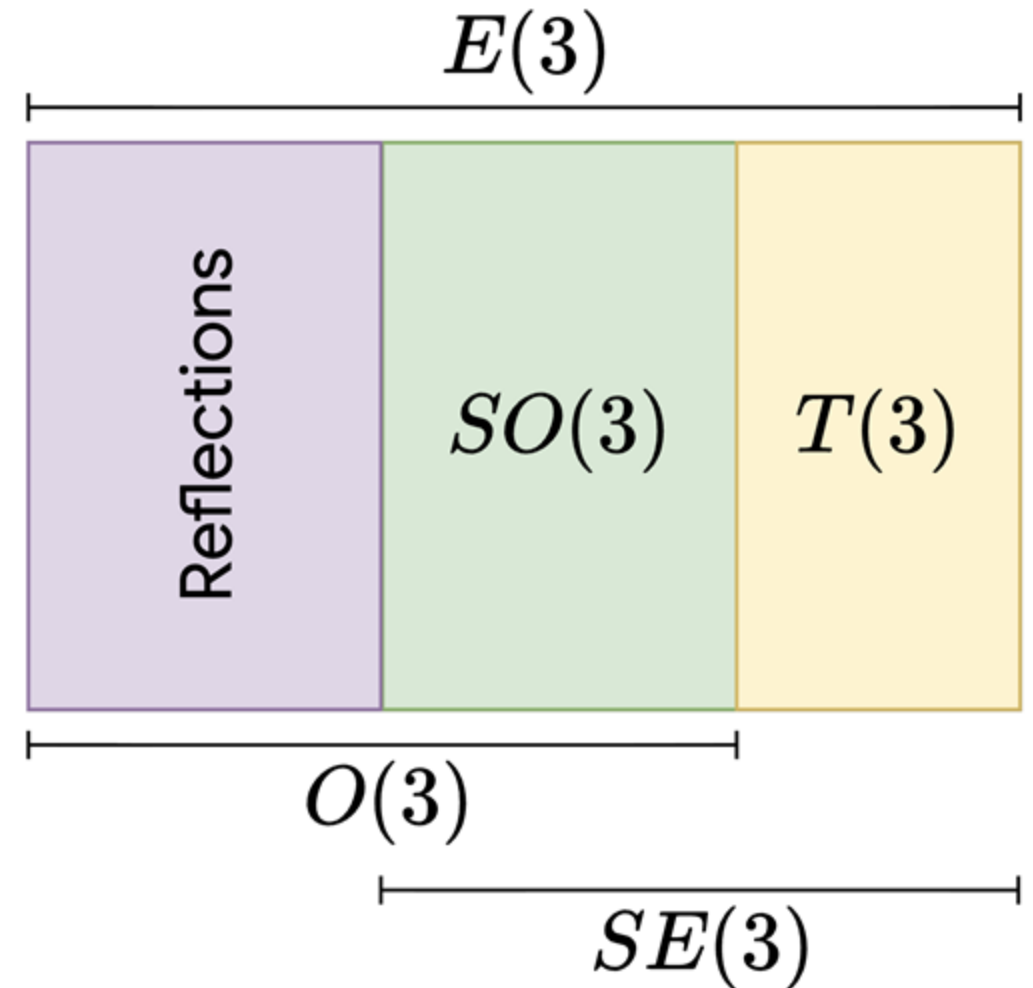
$$T(3) \cup SO(3) \cong SE(3)$$

$$T(3) \cup O(3) \cong E(3)$$

$$SE(3) \subset E(3)$$

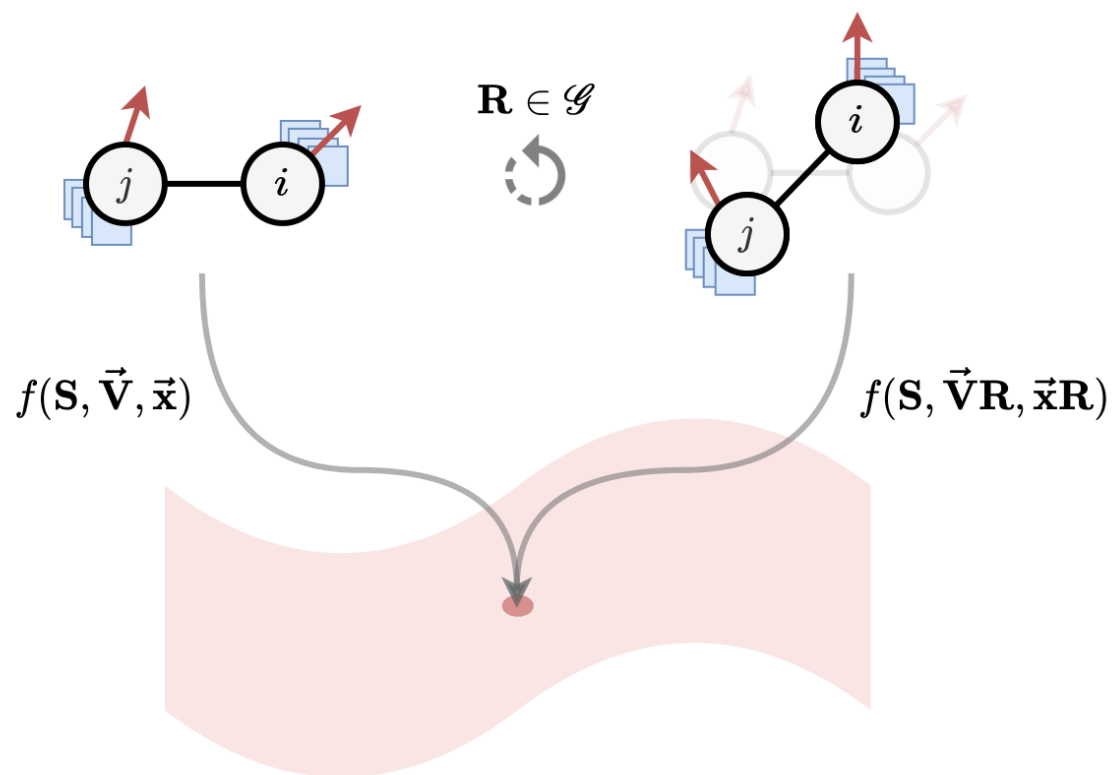
$$SO(3) \subset O(3) \subset E(3)$$

What's the size relationship between the groups?

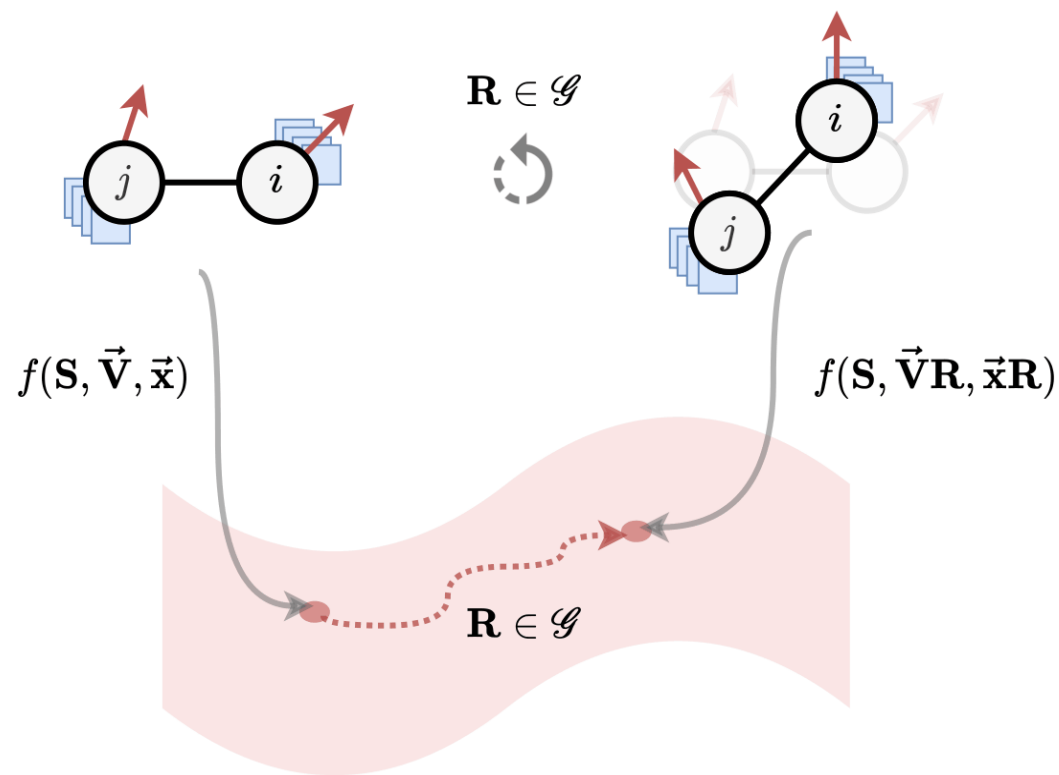


Describing Symmetry via Group Theory

We can restate invariance and equivariance through the lens of groups.
For a 3D rotation $\mathbf{R} \in SO(3)$ acting on a geometric graph,



Invariance to rotations



Equivariance to rotations

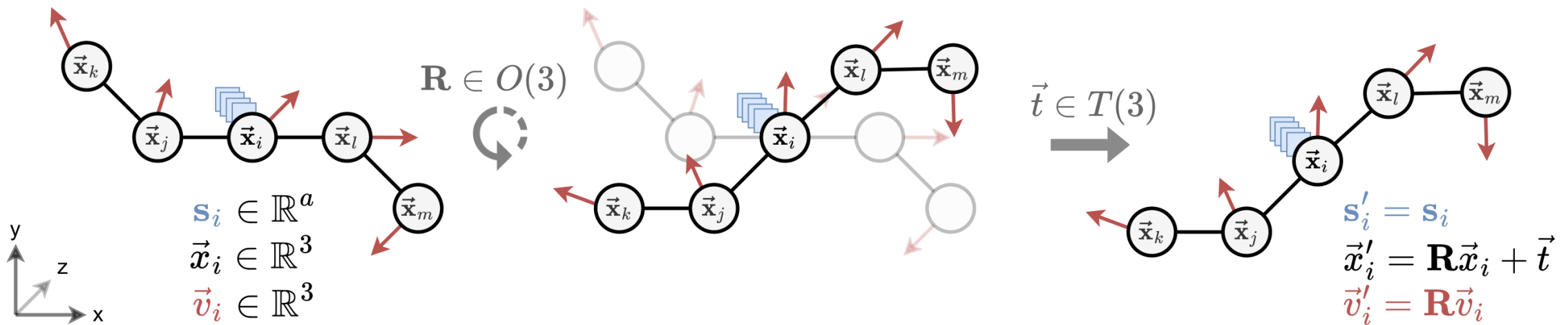
Behavior of Geometric Attributes

Scalar features $\mathbf{S}$ stay invariant to any rigid-body transformations.

Vector features $\vec{\mathbf{X}}, \vec{\mathbf{V}}$ transform with rigid-body transformations of the system.

→ Rotations and Reflections $\mathbf{R} \in O(3)$ act on $\vec{\mathbf{V}}$ and $\vec{\mathbf{X}}$.

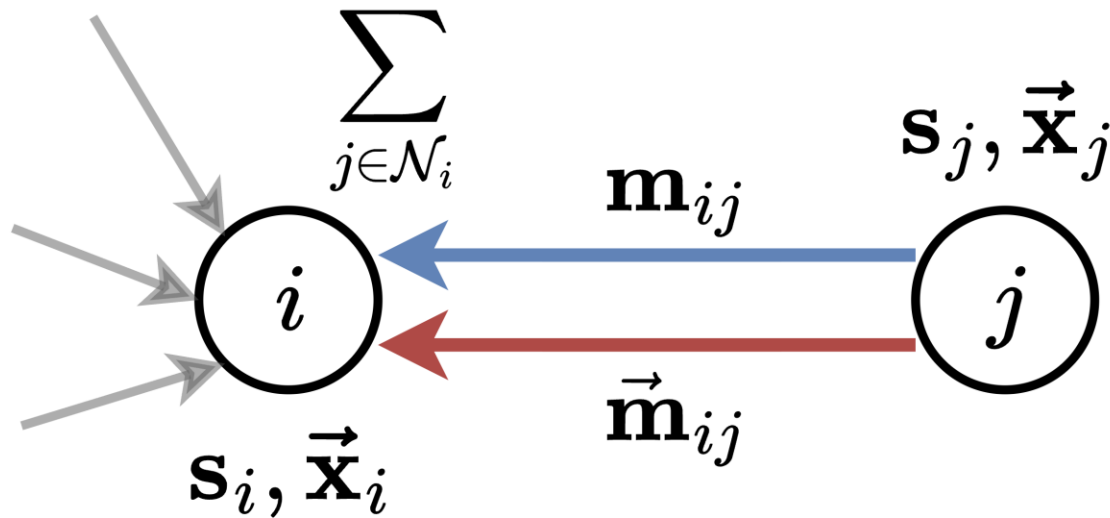
→ Translations $\vec{\mathbf{t}} \in T(3)$ act only on coordinates $\vec{\mathbf{X}}$.



Geometric Message Passing

Scalar features must transform invariantly.

Vector features must transform equivariantly.



$$\mathbf{m}_{ij}^{(t)}, \vec{\mathbf{m}}_{ij}^{(t)} = \textit{Agg}(\cdots)$$

$$\mathbf{s}_i^{(t+1)}, \vec{\mathbf{x}}_i^{(t+1)} = \textit{Upd}(\cdots)$$

Different geometric GNNs recipes formulate **Agg** and **Upd** differently.

Design Space of Geometric GNNs

$E(n)$ -Equivariant GNN [Satorras et al., 2021]

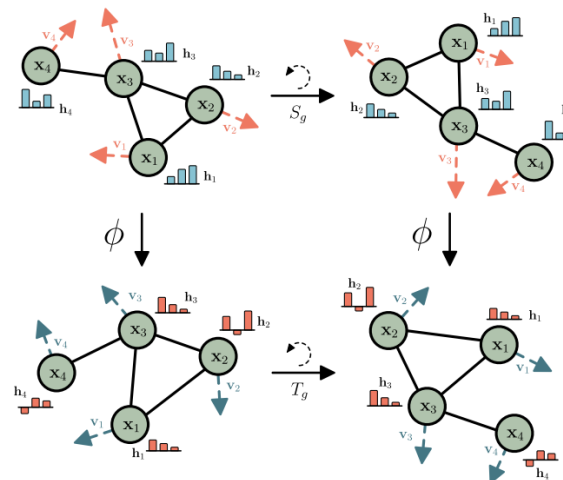
E-GNN offers a simple geometric message passing formulation by having **two separate tracks** for scalar and vector features.

$E(n)$ Equivariant Graph Neural Networks

Victor Garcia Satorras¹ Emiel Hoogetboom¹ Max Welling¹

Abstract

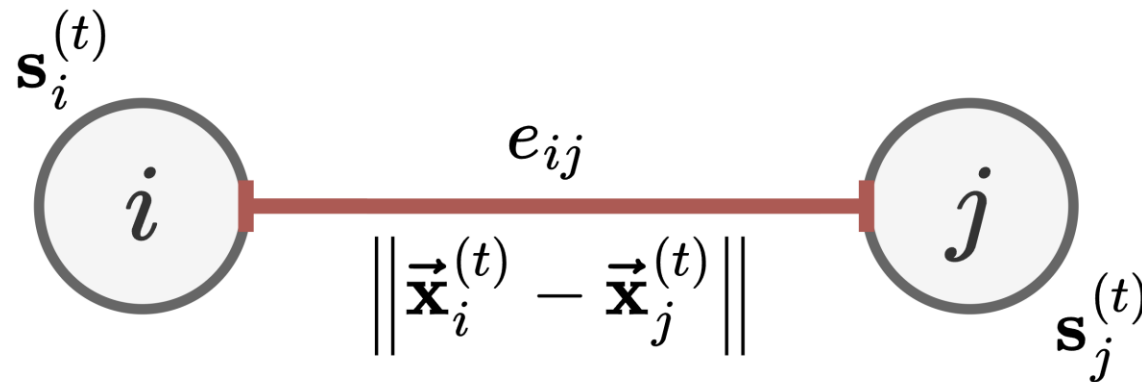
This paper introduces a new model to learn graph neural networks equivariant to rotations, translations, reflections and permutations called $E(n)$ -Equivariant Graph Neural Networks (EGNNs). In contrast with existing methods, our work does not require computationally expensive higher-order representations in intermediate layers while it still achieves competitive or better performance. In addition, whereas existing methods are limited to equivariance on 3 dimensional spaces, our model is easily scaled to higher-dimensional spaces. We demonstrate the effectiveness of our method on dynamical systems modelling, representation learning in graph autoencoders and predicting molecular properties.



E-GNN Message Construction [Satorras et al., 2021]

1. A message is constructed using scalar invariant quantities like node features $\mathbf{s}_i^{(t)}, \mathbf{s}_j^{(t)}$; inter-node distance $\|\vec{\mathbf{x}}_i^{(t)} - \vec{\mathbf{x}}_j^{(t)}\|^2$; and optional edge features e_{ij} .

$$\vec{\mathbf{m}}_{ij} = f_{\theta}(\mathbf{s}_i^{(t)}, \mathbf{s}_j^{(t)}, \|\vec{\mathbf{x}}_i^{(t)} - \vec{\mathbf{x}}_j^{(t)}\|^2, e_{ij})$$



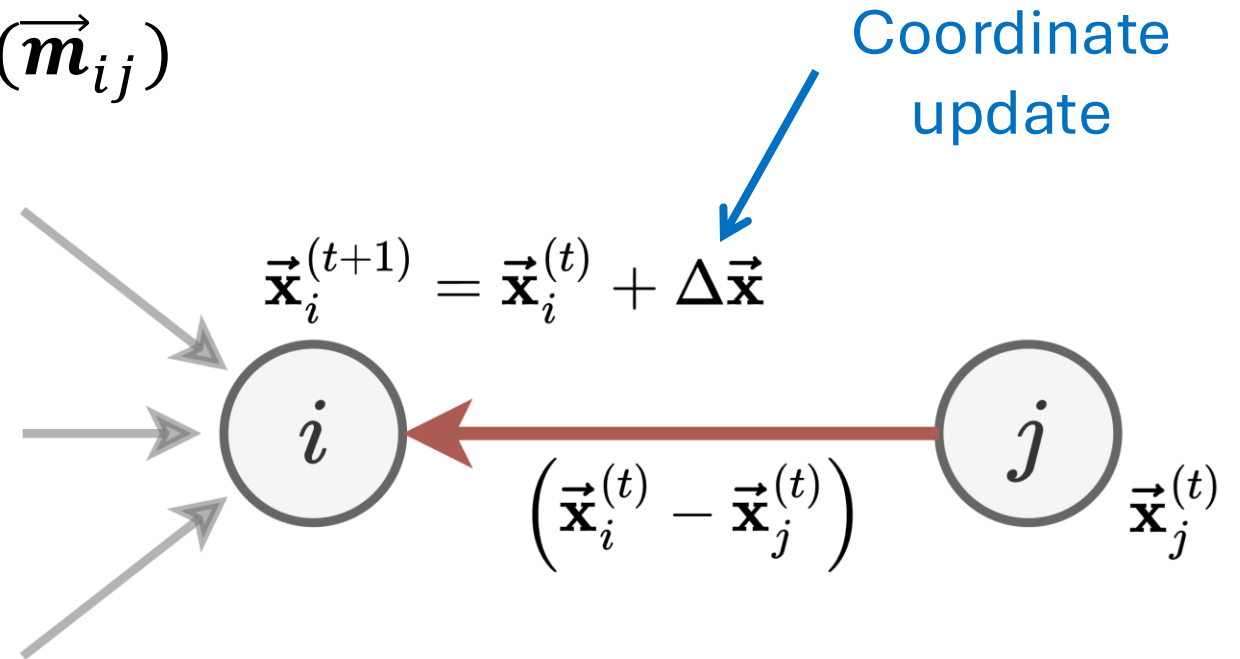
f_{θ} is an MLP.

E-GNN Vector Updates [Satorras et al., 2021]

2. Vector features (like coordinates $\vec{x}$) are updated using the current position and a term that depends on **relative direction** to neighbors. This captures orientation information between nodes.

$$\vec{x}_i^{(t+1)} = \vec{x}_i^{(t)} + \sum_{j \in \mathcal{N}_i} \underbrace{\left(\vec{x}_i^{(t)} - \vec{x}_j^{(t)} \right)}_{\text{Relative direction}} f_{\phi}(\vec{m}_{ij})$$

f_{ϕ} is a MLP that takes in incoming vector messages.



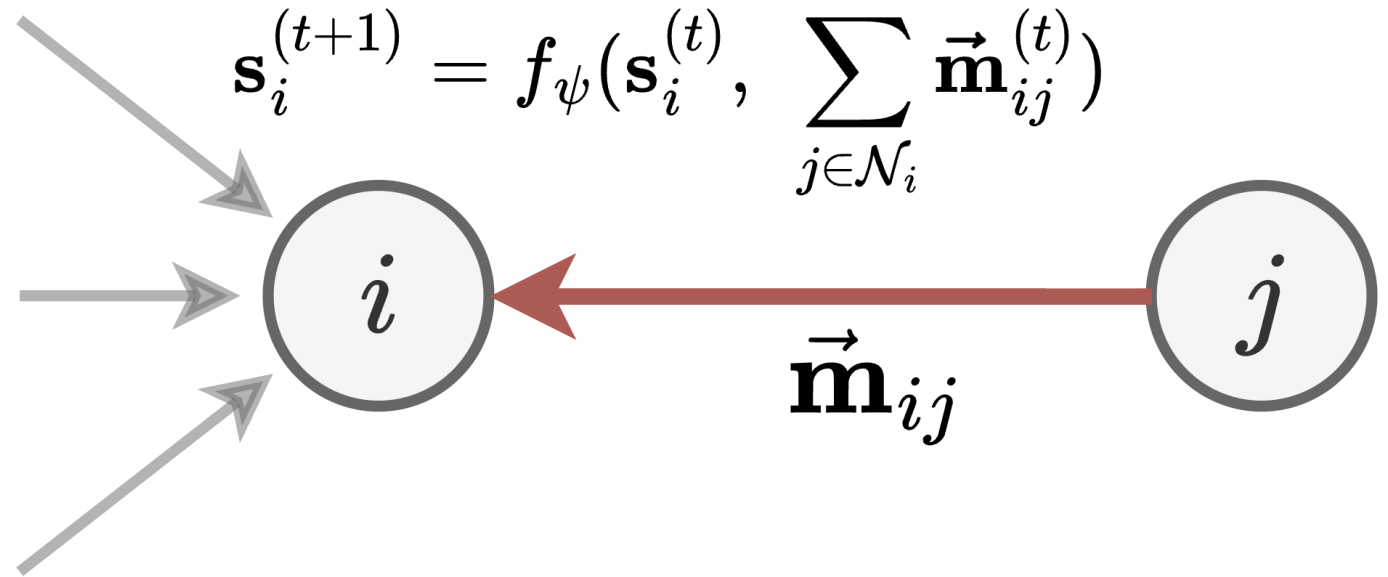
E-GNN Scalar Updates [Satorras et al., 2021]

3. Scalar embeddings are updated collectively using the messages from neighboring nodes and the aggregated vector messages.

$$\overrightarrow{\mathbf{m}}_i = \sum_{j \in \mathcal{N}_i} \overrightarrow{\mathbf{m}}_{ij}$$

$$\mathbf{s}_i^{(t+1)} = f_\psi(\mathbf{s}_i^{(t)}, \overrightarrow{\mathbf{m}}_i^{(t)})$$

f_ψ is a MLP that takes in
vector messages and
current node embedding



Final $E(n)$ -Equivariant Layer [Satorras et al., 2021]

E-GNN offers a simple geometric message passing formulation by having **two separate tracks** for scalar and vector features.

$$\vec{m}_{ij} = f_{\theta}(s_i^{(t)}, s_j^{(t)}, \|\vec{x}_i^{(t)} - \vec{x}_j^{(t)}\|^2, e_{ij})$$

$$\vec{x}_i^{(t+1)} = \vec{x}_i^{(t)} + \sum_{j \in \mathcal{N}_i} (\vec{x}_i^{(t)} - \vec{x}_j^{(t)}) f_{\phi}(\vec{m}_{ij}) \longleftarrow \text{Update vector features (ie, coordinates)}$$

$$\vec{m}_i = \sum_{j \in \mathcal{N}_i} \vec{m}_{ij}$$

$$s_i^{(t+1)} = f_{\psi}(s_i^{(t)}, \vec{m}_i^{(t)}) \longleftarrow \text{Update scalar features}$$

$f_{\theta}, f_{\phi}, f_{\psi}$ are different MLPs.

Proof of SO(3) Symmetry

Suppose we rotated our graph G with a 3D rotation $\mathbf{R} \in SO(3)$. Scalar features $\mathbf{s}_i^{(t)}$ stay the same. Node coordinates become $\mathbf{R}\vec{\mathbf{x}}_i^{(t)} \in \mathbb{R}^3$.

→ We want to show the E-GNN layer **transforms scalars invariantly** and **transforms vectors equivariantly**.

$$\mathbf{R}\vec{\mathbf{x}}_i^{(t+1)} = \mathbf{R} \cdot \text{EGNN}(\vec{\mathbf{x}}_i^{(t)}) = \text{EGNN}(\mathbf{R}\vec{\mathbf{x}}_i^{(t)})$$

$$\mathbf{s}_i^{(t+1)} = \text{EGNN}(\mathbf{R}\mathbf{s}_i^{(t)}) = \text{EGNN}(\mathbf{s}_i^{(t)})$$

Group theory fact:

A rotation acting on any scalar is basically the identity, ie, nothing happens to it.

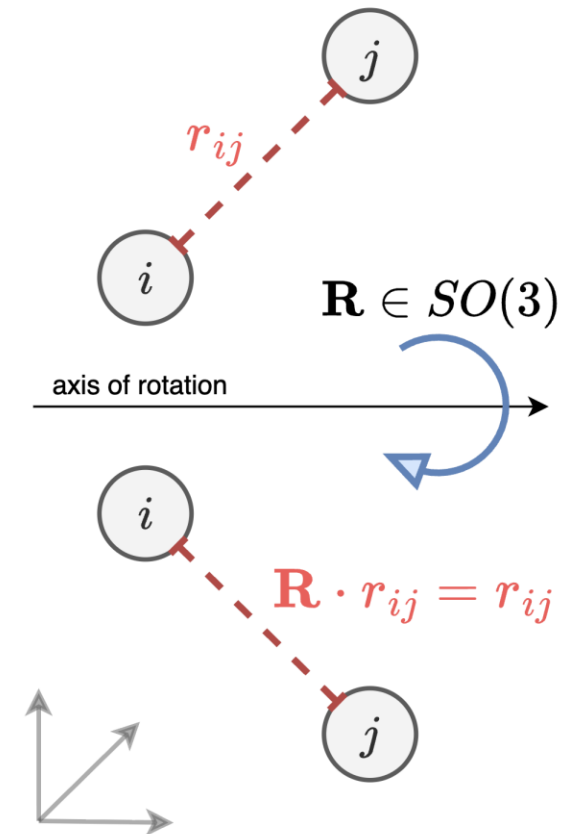
Proof of SO(3)-Equivariance

Suppose we rotated our graph G with a 3D rotation $\mathbf{R} \in SO(3)$. Scalar features $\mathbf{s}_i^{(t)}$ stay the same. Node coordinates become $\mathbf{R}\vec{\mathbf{x}}_i^{(t)} \in \mathbb{R}^3$.

$$\begin{aligned}\vec{\mathbf{m}}_{ij} &= f_{\theta}(\mathbf{s}_i^{(t)}, \mathbf{s}_j^{(t)}, ||\mathbf{R}\vec{\mathbf{x}}_i^{(t)} - \mathbf{R}\vec{\mathbf{x}}_j^{(t)}||^2, e_{ij}) \\ &= f_{\theta}(\mathbf{s}_i^{(t)}, \mathbf{s}_j^{(t)}, ||\vec{\mathbf{x}}_i^{(t)} - \vec{\mathbf{x}}_j^{(t)}||^2, e_{ij})\end{aligned}$$

Rotations do not
affect distances!!

The incoming message $\vec{\mathbf{m}}_{ij}$ from a neighbor j is unaffected by rotations since it's a function of scalar invariant quantities.



Proof of SO(3)-Equivariance

Suppose we rotated our graph G with a 3D rotation $\mathbf{R} \in SO(3)$. Scalar features $\mathbf{s}_i^{(t)}$ stay the same. Node coordinates become $\mathbf{R}\vec{\mathbf{x}}_i^{(t)} \in \mathbb{R}^3$.

$$\begin{aligned}\vec{\mathbf{x}}_i^{(t+1)} &= \mathbf{R}\vec{\mathbf{x}}_i^{(t)} + \sum_{j \in \mathcal{N}_i} \left(\mathbf{R}\vec{\mathbf{x}}_i^{(t)} - \mathbf{R}\vec{\mathbf{x}}_j^{(t)} \right) f_\phi(\vec{\mathbf{m}}_{ij}) \quad \longleftarrow \vec{\mathbf{m}}_{ij} \text{ is unaffected by rotations} \\ &= \mathbf{R}\vec{\mathbf{x}}_i^{(t)} + \sum_{j \in \mathcal{N}_i} \mathbf{R} \left(\vec{\mathbf{x}}_i^{(t)} - \vec{\mathbf{x}}_j^{(t)} \right) f_\phi(\vec{\mathbf{m}}_{ij}) \quad \longleftarrow \text{Rotation matrix } \mathbf{R} \text{ factors out nicely.} \\ &= \mathbf{R} \left(\vec{\mathbf{x}}_i^{(t)} + \sum_{j \in \mathcal{N}_i} \left(\vec{\mathbf{x}}_i^{(t)} - \vec{\mathbf{x}}_j^{(t)} \right) f_\phi(\vec{\mathbf{m}}_{ij}) \right) \quad \longleftarrow \text{Rotation matrix } \mathbf{R} \text{ factors out nicely.}\end{aligned}$$

Proof of SO(3)-Equivariance

Suppose we rotated our graph G with a 3D rotation $\mathbf{R} \in SO(3)$. Scalar features $\mathbf{s}_i^{(t)}$ stay the same. Node coordinates become $\mathbf{R}\vec{\mathbf{x}}_i^{(t)} \in \mathbb{R}^3$.

$$\vec{\mathbf{x}}_i^{(t+1)} = \mathbf{R} \left(\vec{\mathbf{x}}_i^{(t)} + \sum_{j \in \mathcal{N}_i} \left(\vec{\mathbf{x}}_i^{(t)} - \vec{\mathbf{x}}_j^{(t)} \right) f_\phi(\vec{\mathbf{m}}_{ij}) \right)$$

This is basically the output of
a regular EGNN layer!

$$\Rightarrow \mathbf{R} \cdot \text{EGNN}(\vec{\mathbf{x}}_i^{(t)}) = \text{EGNN}(\mathbf{R}\vec{\mathbf{x}}_i^{(t)})$$

Same proof for
reflections, $\mathbf{R} \in O(3)$.

Our equivariance
constraint for vector
quantities is satisfied!

Proof of SO(3)-Equivariance

Suppose we rotated our graph G with a 3D rotation $\mathbf{R} \in SO(3)$. Scalar features $\mathbf{s}_i^{(t)}$ stay the same. Node coordinates become $\mathbf{R}\vec{\mathbf{x}}_i^{(t)} \in \mathbb{R}^3$.

$$\vec{\mathbf{m}}_i = \sum_{j \in \mathcal{N}_i} \vec{\mathbf{m}}_{ij} \quad \leftarrow \text{Final aggregated message unaffected by rotation } \mathbf{R}$$

$$\mathbf{s}_i^{(t+1)} = f_{\psi}(\mathbf{s}_i^{(t)}, \vec{\mathbf{m}}_i^{(t)})$$

$$\Rightarrow \text{EGNN}(\mathbf{R}\mathbf{s}_i^{(t)}) = \text{EGNN}(\mathbf{s}_i^{(t)})$$

Same proof for reflections, $\mathbf{R} \in O(3)$.

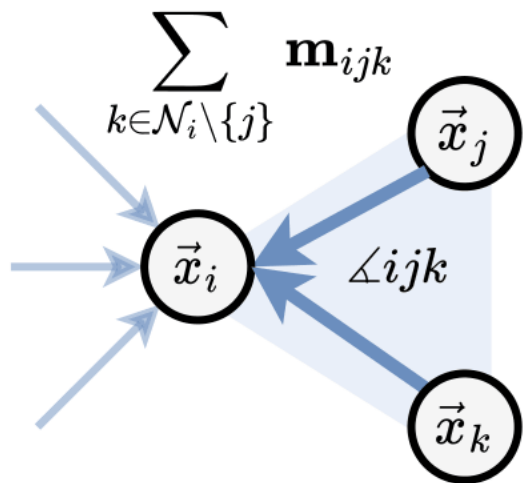
Our invariance constraint for scalar quantities is satisfied!

DimeNet [Gasteiger et al., 2020]

Geometric GNNs can be made **more expressive** by increasing the **body order** of the message passing, ie, by including many-body invariants.

Eg: angular information is important for molecular modeling.

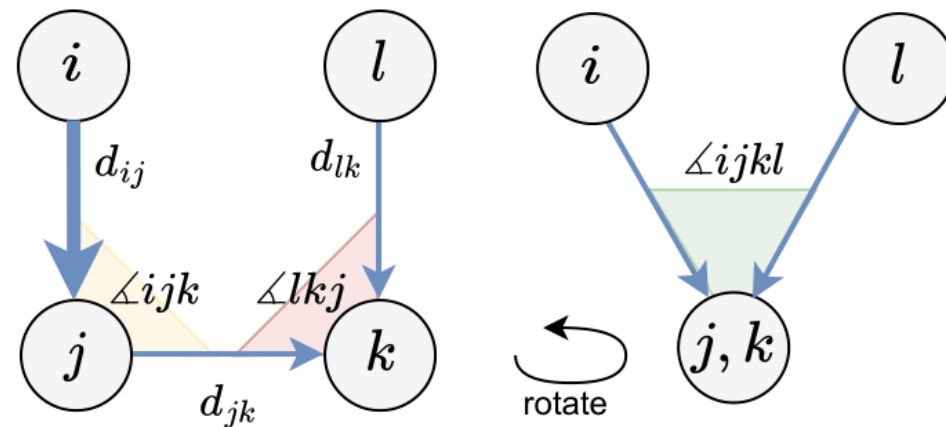
$$\vec{m}_{ij} = f_{\theta}(\mathbf{s}_i^{(t)}, \mathbf{s}_j^{(t)}, \|\vec{x}_i^{(t)} - \vec{x}_j^{(t)}\|^2, e_{ij}, \angle ijk)$$



Incorporating n -body interactions help GNNs learn local geometry better.

$$\mathbf{m}_{ijk} = f(\mathbf{s}_j, \mathbf{s}_k, \|\vec{x}_{ij}\|, \angle ijk)$$

GemNet [Gasteiger et al., 2024] uses dihedral angles between quadruplets.



Radial Basis Functions

In geometric graphs, edges do not carry binary meaning.

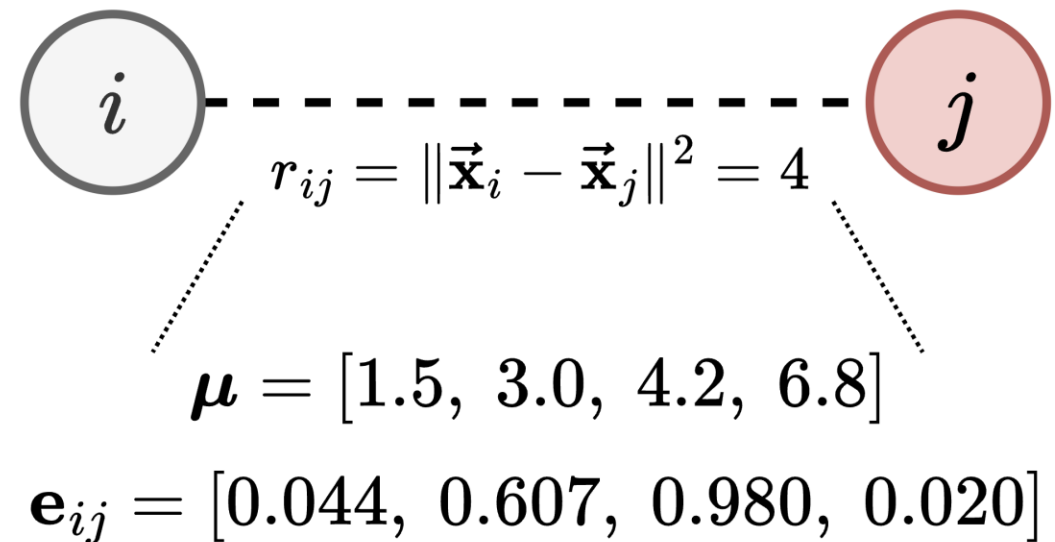
RBFs convert a scalar distance $r_{ij} = \|\vec{\mathbf{x}}_i^{(t)} - \vec{\mathbf{x}}_j^{(t)}\|^2$ into a feature vector

Each basis function asks, “*how close is this distance to a reference value μ_k ?*”

$$\phi_k(r) = \exp\left(-\frac{(r - \mu_k)^2}{2\sigma^2}\right)$$

$$\mathbf{e}_{ij} = [\phi_1(r_{ij}), \dots, \phi_K(r_{ij})]$$

RBFs convert scalar distances into a **soft histogram** of reference distances.



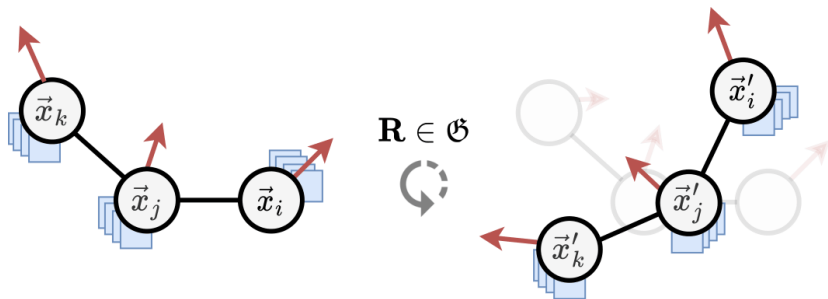
Intuition:

Geometric Message Passing “transports” vector information across the graph in the same way standard GNNs propagate information across their receptive field.

Unconstrained Approaches

- Enforcing strict symmetries is computationally expensive.
- Training such models takes a long time, depending on application.
- Recent work shows (in/eq)uivariance can be *learned* very quickly.
- Approximately-(in/eq)uivariant models are a **favorable middle-ground**.

Data Augmentations



Regularization



Relaxed Architectures

$$\Sigma \left\{ \begin{array}{c} \text{3D coordinate system} \\ R_1 \end{array} \rightarrow \begin{array}{c} \text{3D coordinate system} \\ R_2 \end{array} \cdots \begin{array}{c} \text{3D coordinate system} \\ R_{|G|} \end{array} \right\}$$

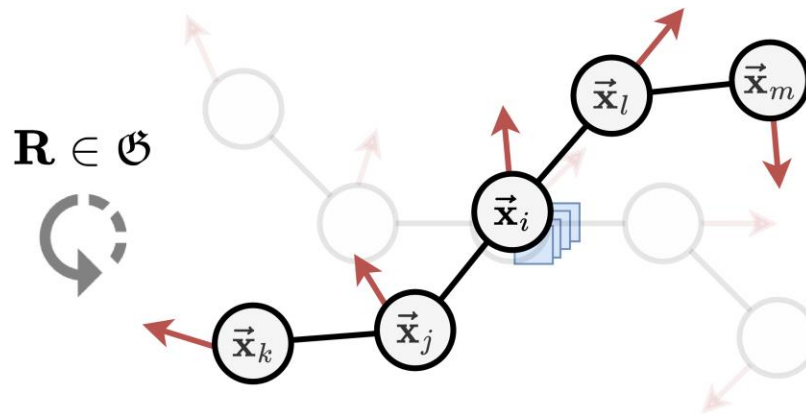
Group Theory Fact:

Unconstrained methods all rely on the fundamental principle of *Group Averaging*, which allows us to *approximate* symmetry-awareness to any target group (eg: $SO(3)$).

Group Averaging

Given an unconstrained function f , you can create an (in/eq)uivariant function based on f by **averaging over** elements $g \in \mathcal{F} \subseteq SO(3)$:

$$\bar{f}(x) = \frac{1}{|\mathcal{F}|} \sum_{g \in \mathcal{F}} f(g \cdot x)$$



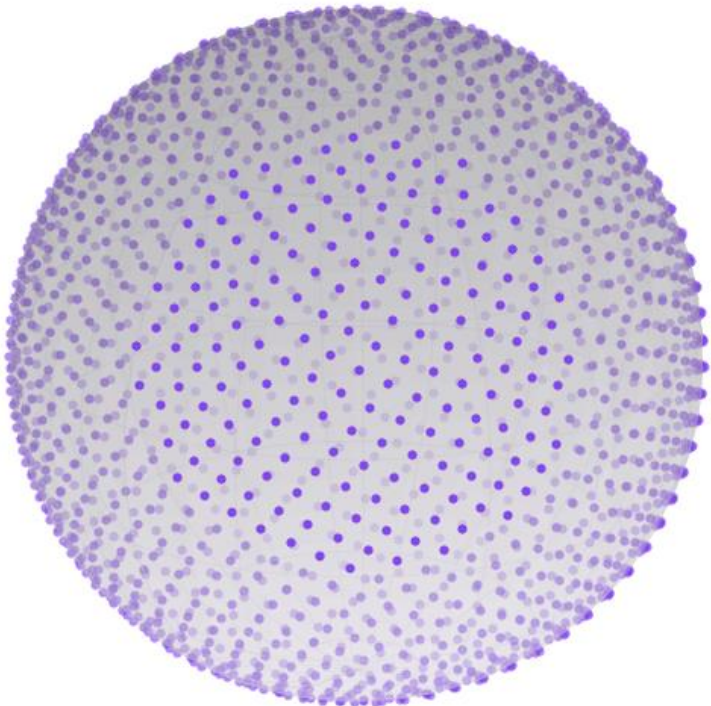
The larger $|\mathcal{F}|$ is, the more accurate the approximation.

For continuous groups like $SO(3)$, you can uniformly sample a subset of rotations and average over it.

This means you no longer need f to be (in/eq)uivariant (ie, it can be a vanilla Transformer with data augmentations).

Group Averaging Intuition

The approximation error decreases between a fully-invariant model and approximately-invariant model as the **coverage** of $SO(3)$ increases.



Suppose this sphere represents the group, $SO(3)$.

Each **purple point** is a valid 3D rotation by some angle about some axis.

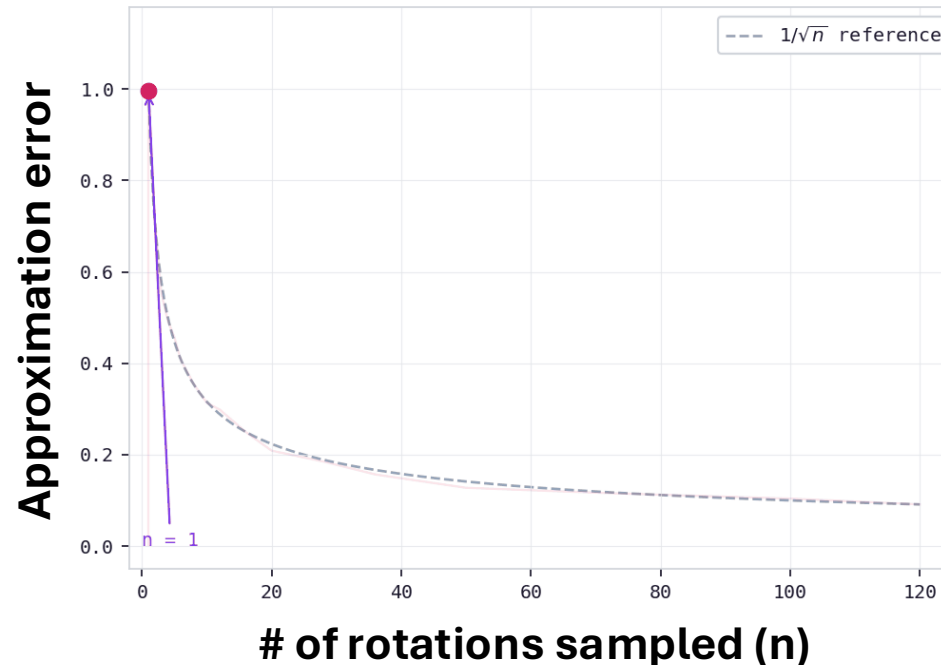
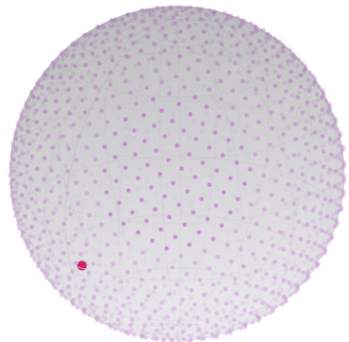
Eg: A fully-equivariant model f_θ is equivariant to ANY rotation from this group:

$$f_\theta(\mathbf{S}, \vec{\mathbf{X}}\mathbf{R}^\top) = f_\theta(\mathbf{S}, \vec{\mathbf{X}})\mathbf{R}^\top$$

Group Averaging Intuition

The approximation error decreases between a fully-invariant model and approximately-invariant model as the **coverage** of $SO(3)$ increases.

The more we sample random rotations from $SO(3)$, the more the **approximation error decreases**, giving us an approx-invariant backbone.



Each **red point** is a valid randomly-sampled rotation.

The model's inputs are **averaged** over these rotations, $\mathbf{R}$.

$$f_{\theta}(\mathbf{S}, \vec{\mathbf{X}}\mathbf{R}^{\top}) \approx f_{\theta}(\mathbf{S}, \vec{\mathbf{X}})\mathbf{R}^{\top}$$

[defn.] *Frame*

The subset of elements (eg: rotations) sampled to average over a group $\mathcal{G}$ (eg: $SO(3)$) is called a frame, $\mathcal{F} \subseteq \mathcal{G}$.

→ Invariance/Equivariance to elements in $\mathcal{F}$
guarantees approximate invariance/equivariance to $\mathcal{G}$.

REMUL: Invariance via Regularization [Elhag et al., 2024]

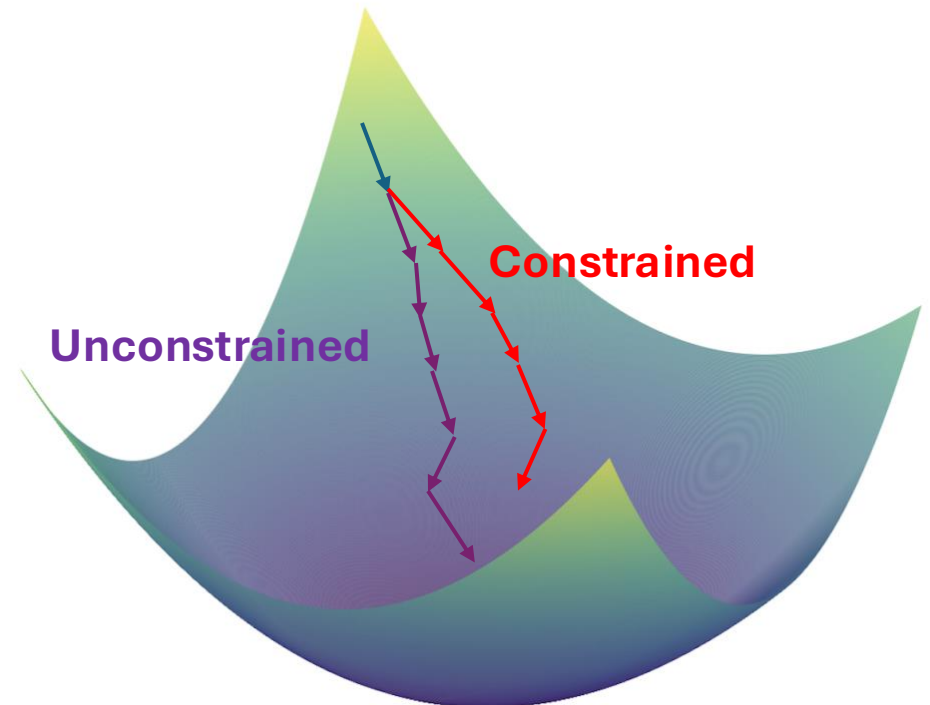
We can use regularize the model's **parameters** to remain approximately equivariant to sampled rotations $\mathbf{R} \in \text{SO}(3)$.

$$\mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^n ||f_{\theta}(\mathbf{R}\vec{x}_i) - \textcolor{red}{y}_i||^2$$

(scalar target)

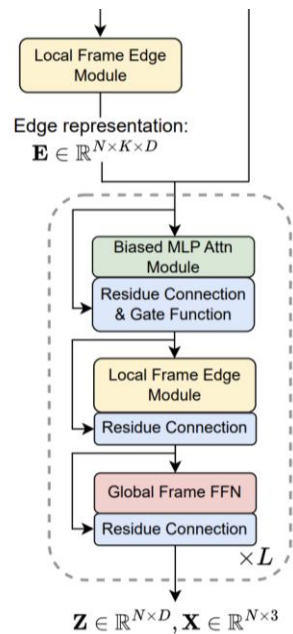
This again leverages *group averaging*, but **in the loss instead of the data**. This can be added to the task's main objective.

Unconstrained models can now approximate equivariant models very well with low error.

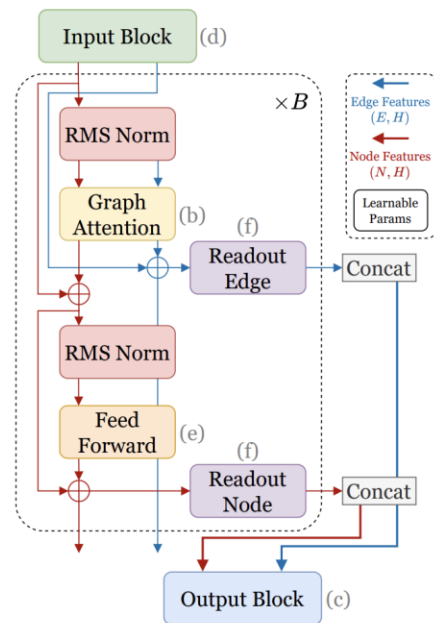


Relaxed Hybrid Architectures

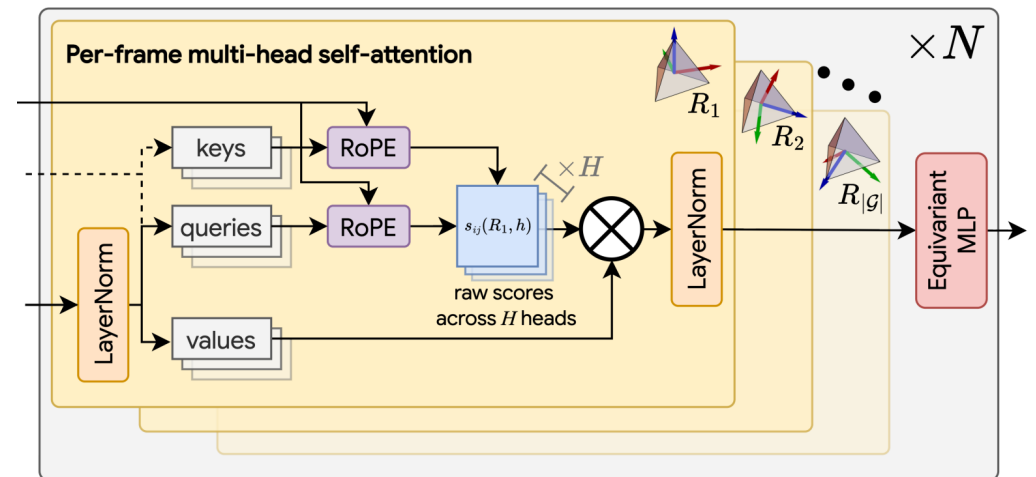
Architectures can enforce symmetry by using *group averaging* on their hidden layer representations **OR** by devising clever ways of choosing the frame $\mathcal{F}$ to average over.



FAE-Net / FAFFormer



EScAIP / AllScAIP



PlatoFormer

FA-GNN [Puny et al., 2023; Duval et al., 2023]

FA-GNN takes a MPNN-like architecture and imbues it with *approximate* SE(3)-invariance via **frame/group averaging**.

FRAME AVERAGING FOR INVARIANT AND EQUIVARIANT NETWORK DESIGN

Omri Puny^{*1} Matan Atzmon^{*1} Heli Ben-Hamu^{*1} Ishan Misra²

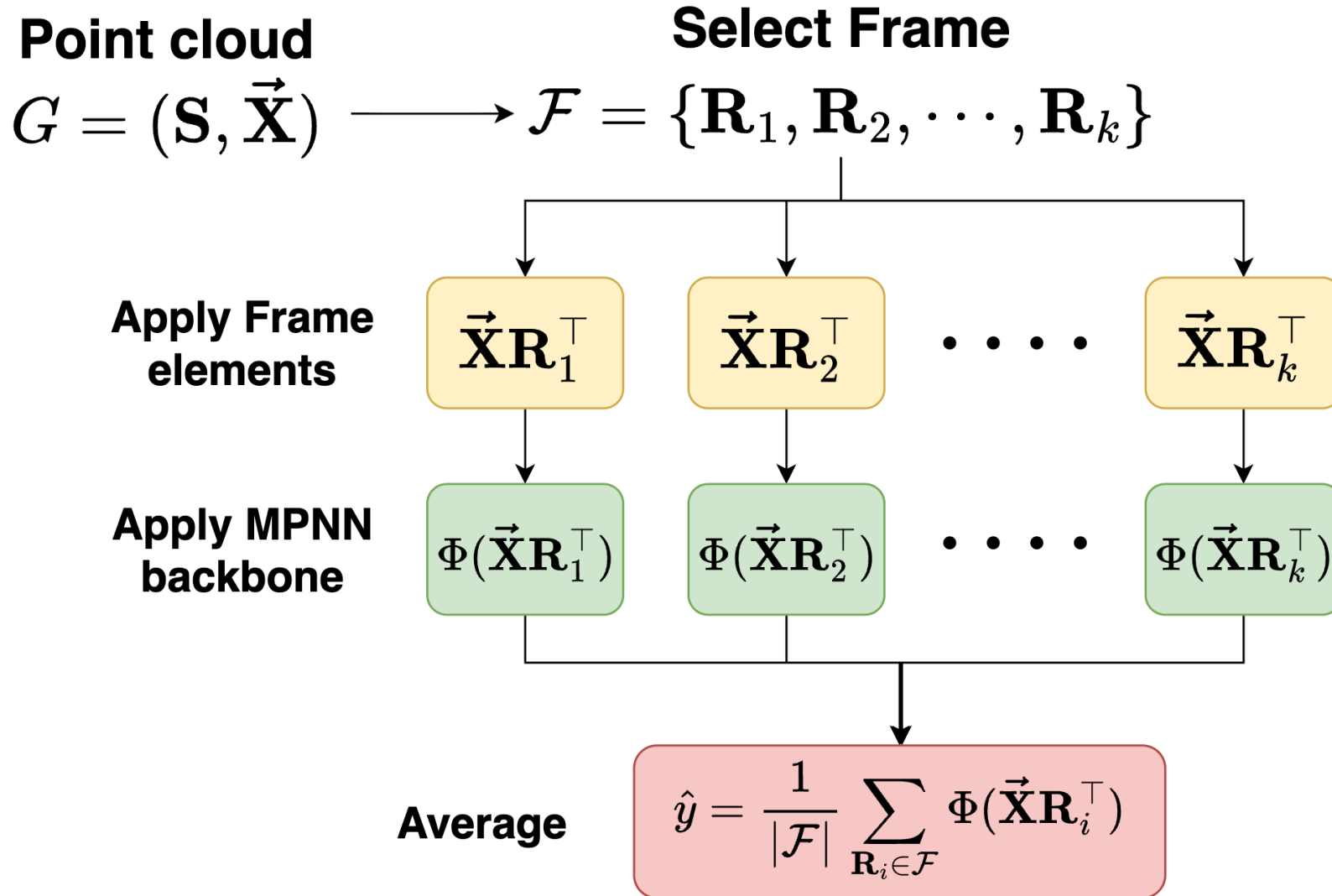
Aditya Grover² Edward J. Smith² Yaron Lipman^{2 1}

¹Weizmann Institute of Science ²Facebook AI Research

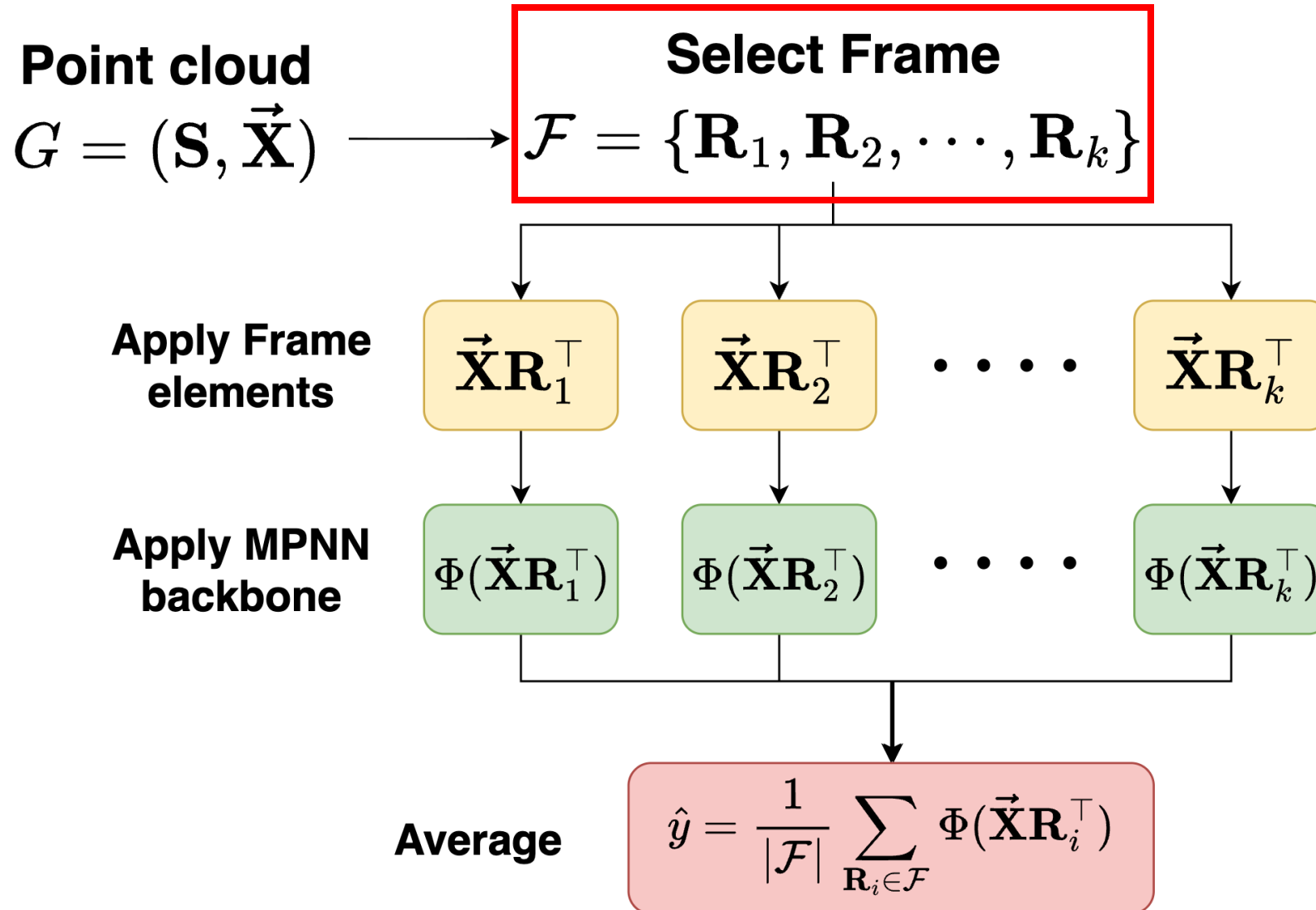
FAENet: Frame Averaging Equivariant GNN for Materials Modeling

Alexandre Duval^{*1 2} Victor Schmidt^{*2} Alex Hernandez Garcia² Santiago Miret³ Fragkiskos D. Malliaros¹
Yoshua Bengio^{2 4} David Rolnick^{2 5}

FA-GNN



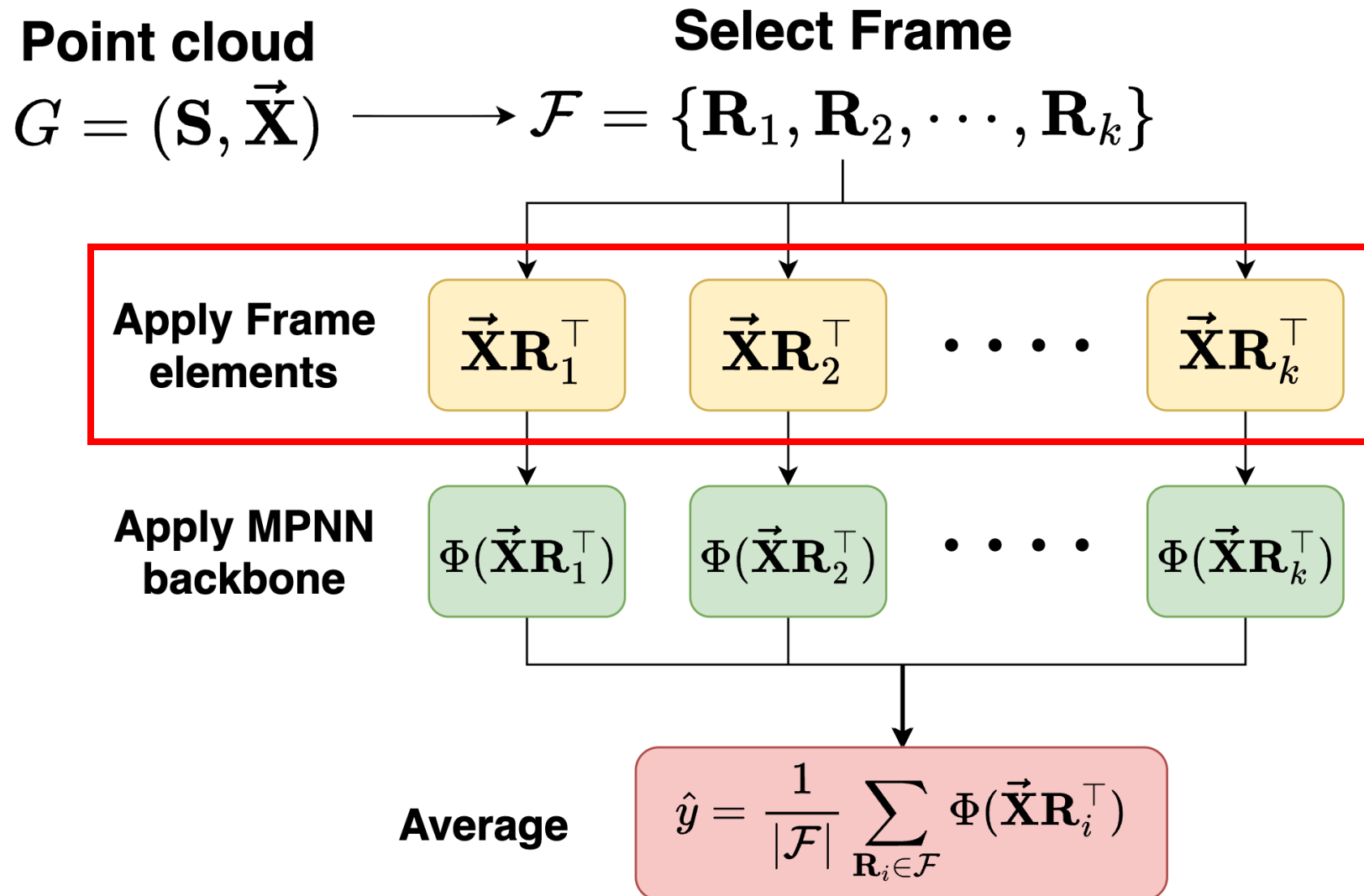
FA-GNN



The frame $\mathcal{F}$ is created by **running PCA** on vector features $\vec{\mathbf{X}}$ (eg: coordinates).

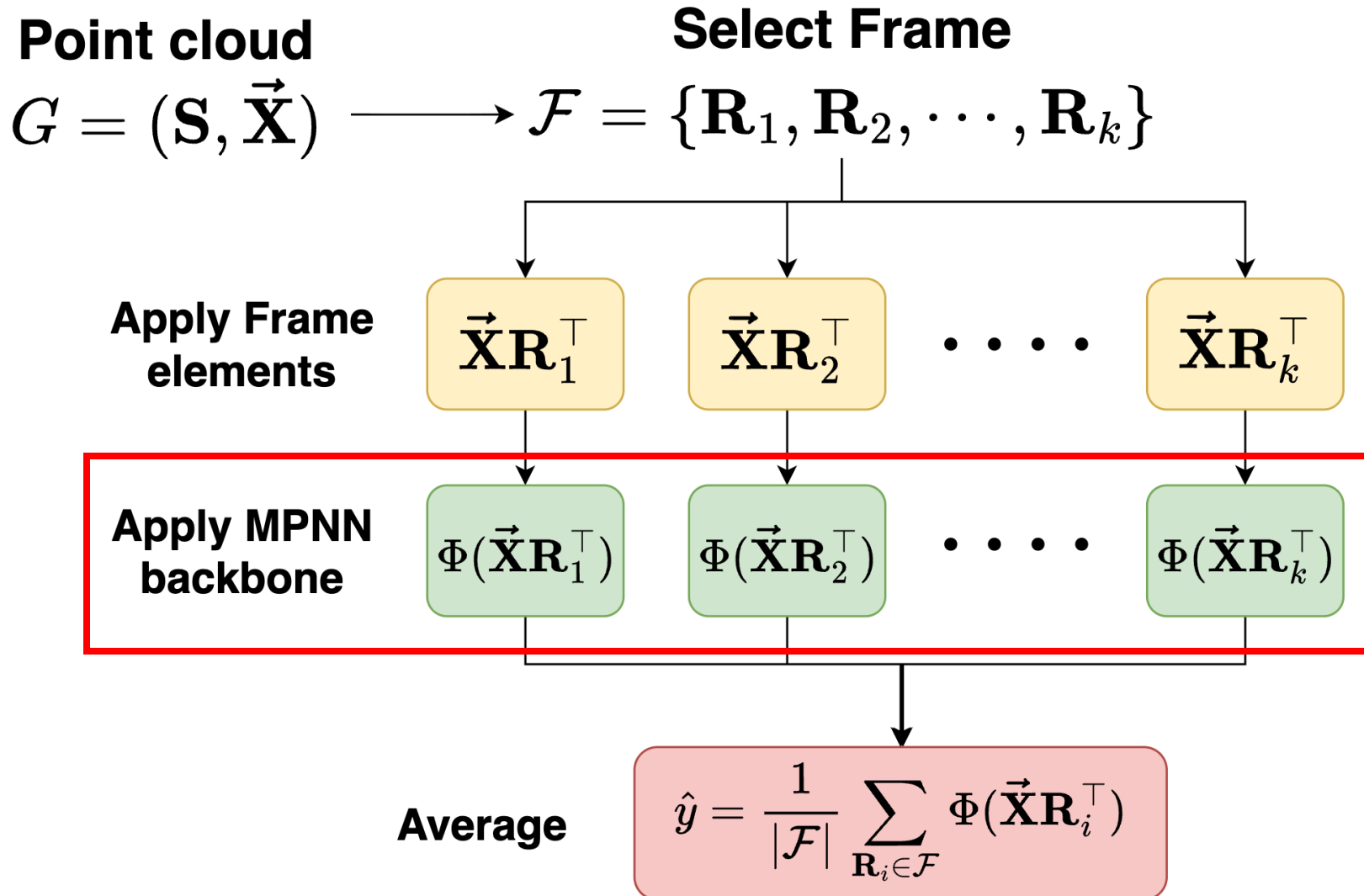
Geometrically, the principal components of $\vec{\mathbf{X}}$ give us the **dominant axes** inherently present in the point cloud.

FA-GNN



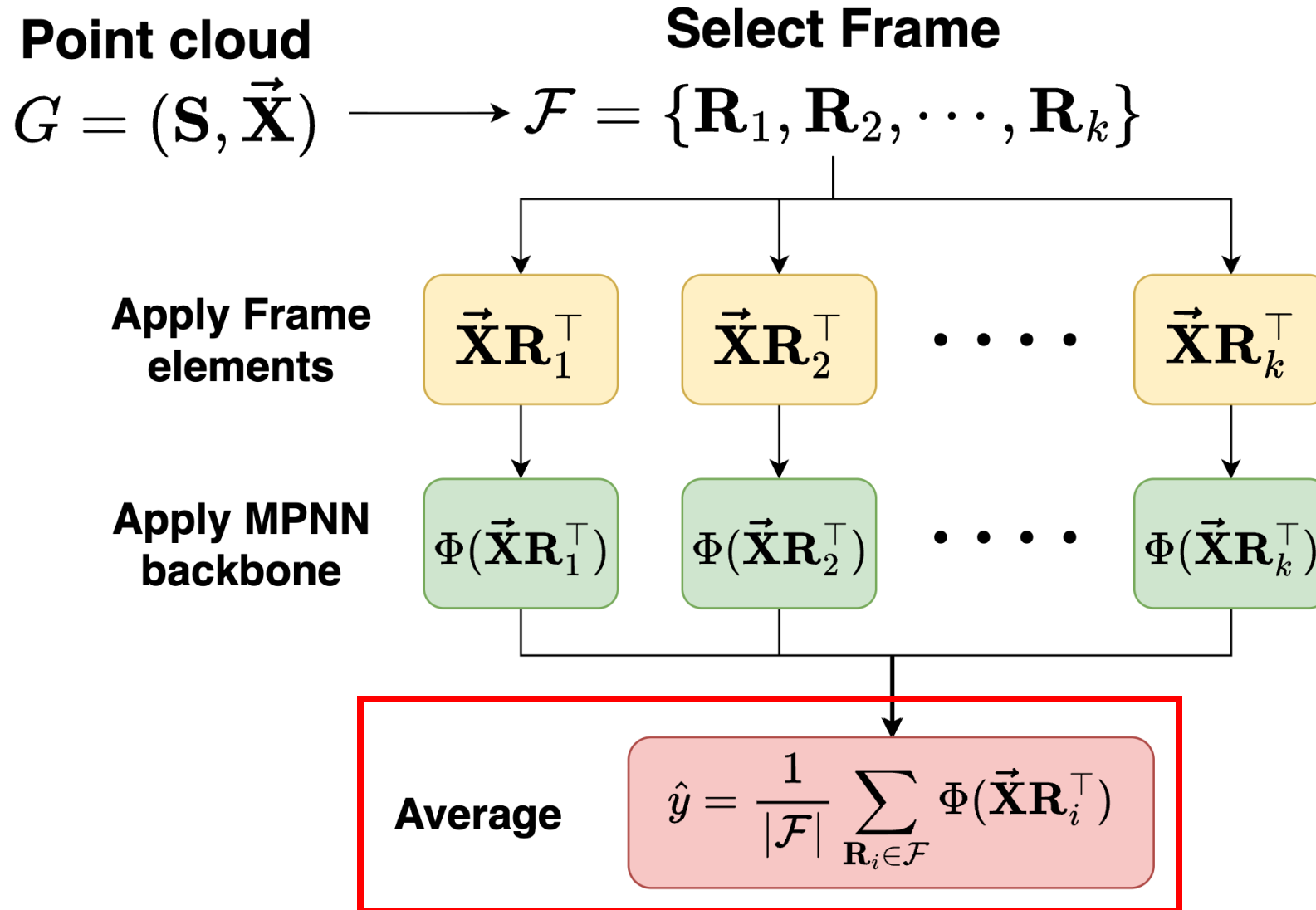
Applying the frame projects $\vec{\mathbf{X}}$ onto the local reference frame defined by rotation $\mathbf{R}_k$.

FA-GNN



Here, the backbone is a vanilla MPNN Φ that is NOT invariant by design, allowing for flexible choice of architecture.

FA-GNN



The final output $\hat{y}$ is approximately invariant to the target group, $\mathcal{G} = SO(3)$.

This is where you use the **group averaging** equation we saw earlier:

$$\bar{f}(x) = \frac{1}{|\mathcal{F}|} \sum_{g \in \mathcal{F}} f(g \cdot x)$$

Platonic Transformers [Islam et al., 2025]

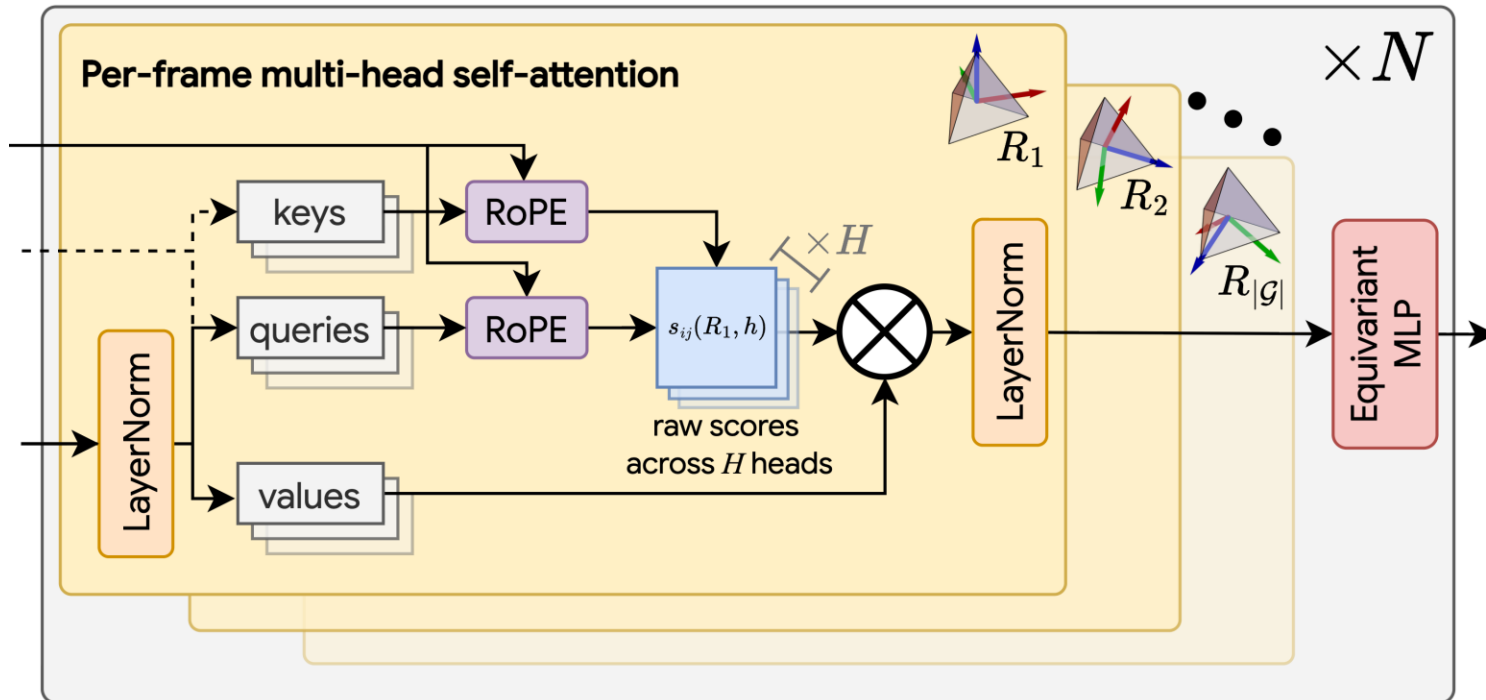
Point cloud

$$G = (\mathbf{S}, \vec{\mathbf{X}})$$

Select Frame

$$\mathcal{F} = \{\mathbf{R}_1, \mathbf{R}_2, \dots, \mathbf{R}_k\}$$

The Platonic Transformer selects its frame $\mathcal{F}$ based on the **symmetry groups of Platonic Solids**. Conveniently, these symmetry groups are subgroups of $O(3)$.



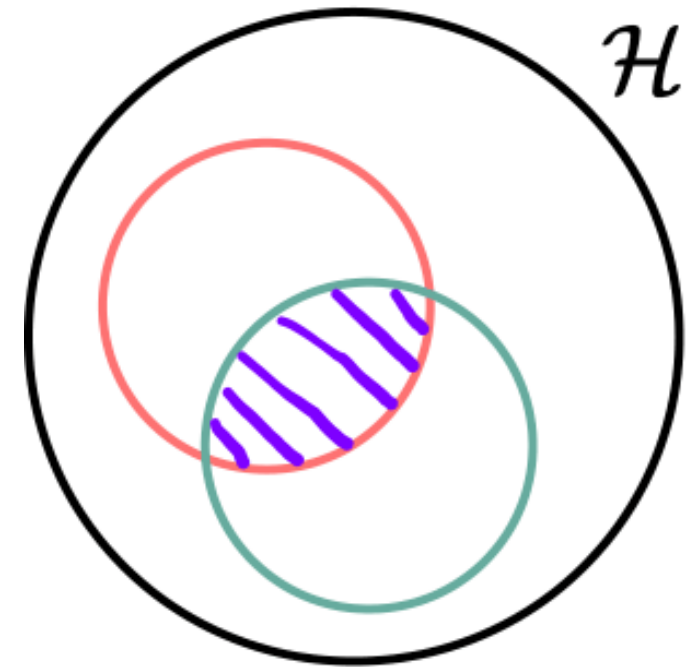
This formulation ensures the self-attention scores are *orientation-aware*, unlike regular self-attention.

Symmetry-enforcing Approaches

Strictly equivariant models can only learn a narrow subset of allowable functions.

Approximately-equivariant models *cut through* disallowed regions of the equivariant hypothesis space $\mathcal{H}$.

Unconstrained models have **smoother loss landscapes** and can be trained faster and for longer than symmetry-aware models.



Non-equivariant

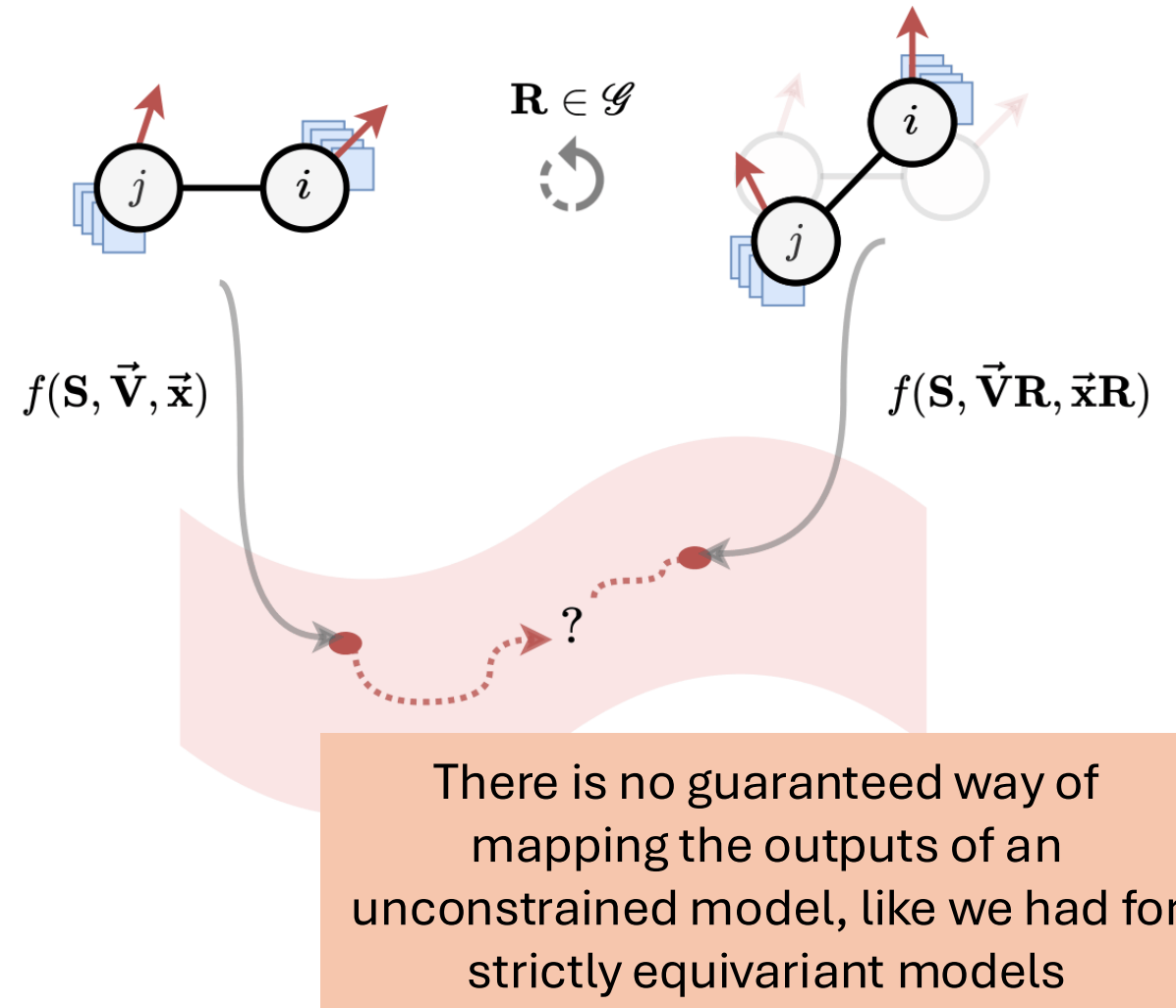
Equivariant

Approx. Equivariant

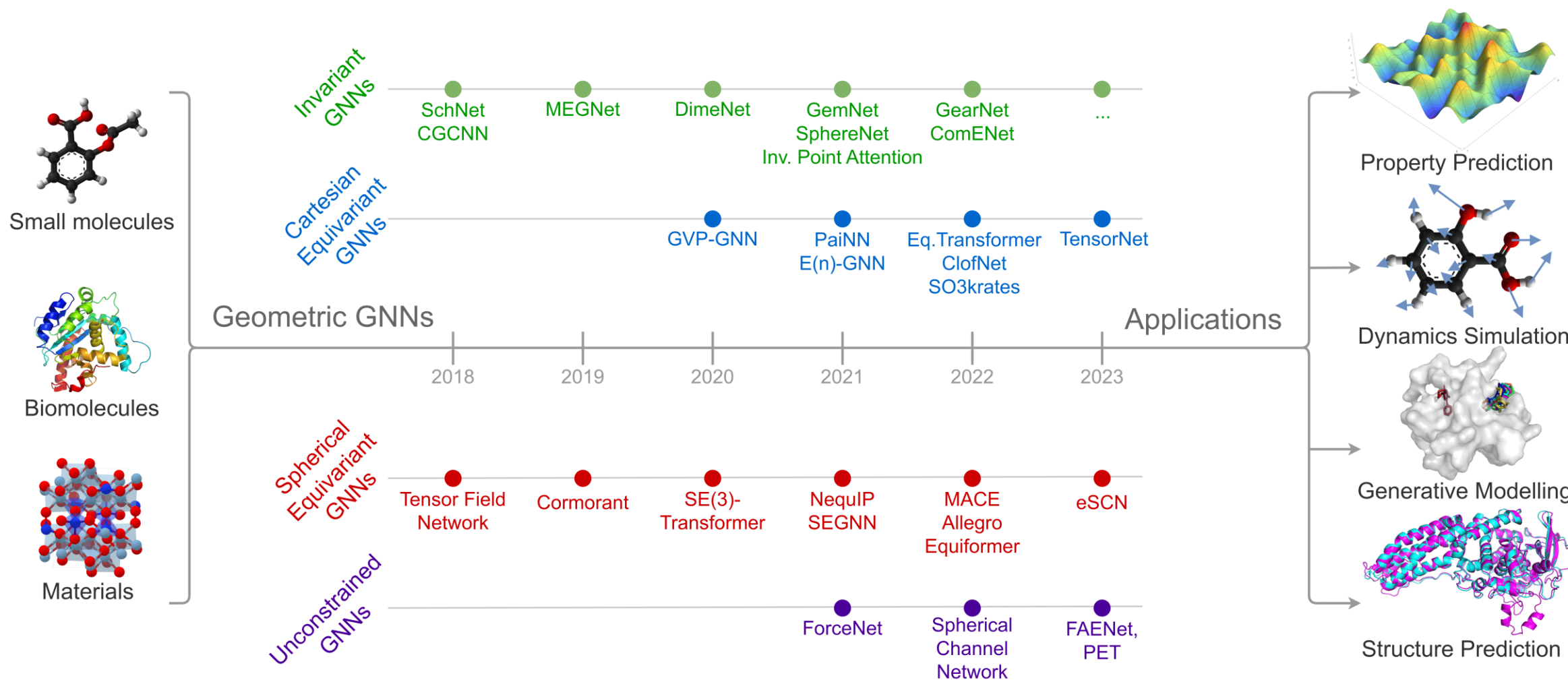
Symmetry-enforcing Approaches

We can train unconstrained models to be approximately equivariant with very low error.

The equality between $f_{\theta}(\mathbf{S}, \vec{\mathbf{X}}, \vec{\mathbf{V}})$ and $f_{\theta}(\mathbf{S}, \mathbf{R}\vec{\mathbf{X}}, \mathbf{R}\vec{\mathbf{V}})$ is no longer theoretically guaranteed but their gap can be optimized and reduced through training.



Geometric GNN Timeline



Content Credits

This lecture presents abridged material originally from the *Hitchhiker's Guide to Geometric GNNs for 3D Atomistic Systems* (arxiv:2312.07511). Figures are used or recreated with permission from the authors, Alexandre Duval, Simon V. Mathis, Chaitanya K. Joshi, Victor Schmidt, and others.

The overall flow is heavily motivated by the *Geometric Deep Learning Proto-Book* by Michael M. Bronstein, Joan Bruna, Taco Cohen, Petar Veličković, as well as Course *L65: Geometric Deep Learning*, taught at the University of Cambridge (UK) Computer Laboratory by Pietro Liò and Petar Veličković.